

Bloque 03_seguridad_web

Índice de módulos

- Modelo de pruebas de API
- Inyección de argumentos y opciones
- Autenticación, autorización y sesiones en APIs
- Controles client-side y validacion del servidor
- Inyección de comandos de sistema operativo
- Enumeracion web con respuestas comodin (wildcard)
- Seguridad de subida de ficheros
- Seguridad de GraphQL
- IDOR / Broken Object Level Authorization (BOLA)
- Seguridad de JWT
- Inclusión local de ficheros (LFI)
- De LFI a ejecucion (RCE)
- Análisis de MFA y OTP
- OAuth, OIDC y CORS
- Inyección SQL (SQLi)
- Server-Side Request Forgery (SSRF)
- Server-Side Template Injection (SSTI)
- Atacar WordPress
- Cross-Site Scripting (XSS)
- XML External Entity (XXE)

Modelo de pruebas de API

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Una API expone contratos HTTP que deben probarse por recurso, método, entrada y contexto de identidad.

Un inventario incompleto hace que se pierdan endpoints administrativos, versiones antiguas o métodos menos visibles.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la aplicación usa JSON, rutas /api, documentación OpenAPI, GraphQL o clientes SPA.

Cómo identificarla

- rutas /api
- respuestas JSON
- OpenAPI/Swagger
- versiones v1/v2

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Identifica al menos un endpoint funcional, su método, parámetros y una respuesta reproducible.

Evidencia esperada: Dispones de un mapa de recursos y de una lista priorizada para probar autenticación y autorización.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.

5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS $URL/swagger.json
```

Objetivo y contexto: Busca un contrato OpenAPI publicado.

Resultado esperado: Listado de rutas, metodos y esquemas.

Capa 2: prueba documentada

```
curl -sS -i -X OPTIONS "$URL/api/resource"
```

Objetivo y contexto: Comprueba metodos aceptados por un recurso.

Resultado esperado: Cabeceras Allow y respuesta del endpoint.

Anatomía de los payloads

La primera prueba es `curl -sS $URL/swagger.json`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Dispones de un mapa de recursos y de una lista priorizada para probar autenticacion y autorizacion.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Descubre rutas y metodos.
- Repite peticiones con entradas controladas.
- Anota diferencias de estado y permisos.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No aporta mucho si solo existe una pagina estatica sin peticiones de datos ni autenticacion.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Mapear endpoints, metodos, parametros y respuestas antes de intentar explotar una API. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(api-testing-model\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Inyección de argumentos y opciones

Objetivos de aprendizaje

Determinar cuándo una entrada crea uno o varios elementos de argv, diferenciar opción de operando y explotar o descartar capacidades del programa sin atribuirles a una shell inexistente.

Prerrequisitos

Procesos, argv, shell splitting, parsers de opciones, -- y documentación de herramientas.

Fundamentos técnicos

Un espacio no crea por sí mismo otro argumento. El componente que construye argv decide:

```
subprocess.run(["curl", user_input])          # un solo argumento
subprocess.run(["curl", *shlex.split(user_input)]) # varios argumentos
subprocess.run("curl " + user_input, shell=True)  # shell: splitting y operadores
```

El primer caso puede permitir controlar un operando completo, pero el espacio sigue dentro de él. El segundo habilita argumentos u opciones adicionales sin gramática de shell. El tercero puede convertirse además en command injection.

Modelo mental

```
entrada -> ¿split explícito? -> argv[] -> parser de opciones -> operación
        -> ¿shell? -----> palabras/operadores -> proceso(s)
```

Superficie de ataque

Wrappers de curl, wget, compresores, convertidores, multimedia, compiladores y cualquier función que exponga opciones de una CLI.

Cómo identificarla

- Error unknown option o cambio inocuo de comportamiento propio de la herramienta.
- ; id es literal, pero una opción adicional reconocida cambia la salida.
- Logs muestran más elementos de argv que el previsto.
- Un operando adicional provoca una segunda operación sin segundo proceso.

Preguntas que debo hacerme

1. ¿Quién divide el texto y con qué reglas?
2. ¿Control un elemento, varios o una posición fija?
3. ¿El programa acepta opciones después de operandos?
4. ¿Implementa -- como fin de opciones?
5. ¿Qué capacidad inocua confirma llegada al parser de opciones?

Prueba mínima

Usar una opción no destructiva documentada para la versión detectada y verificar el argv resultante. Un error de opción confirma parser de opciones, no ejecución de código.

Construcción progresiva del payload

1. Entrada válida y salida normal.
2. Un valor con espacio y log argv.
3. Opción inocua, por ejemplo cambiar verbosidad o cabeceras, solo si se crea un elemento adicional.

4. Operando adicional hacia un listener propio.
5. Comparar con -- y con un único argumento literal.

Anatomía de los payloads

- Contexto de entrada: campo resources de un importador.
- Sintaxis original: `subprocess.run(["curl", *shlex.split(resources)])`.
- Entrada controlada: lista textual dividida por `shlex.split`.
- Transformaciones conocidas: JSON decode y splitting POSIX.
- Parser final: parser de opciones de curl.
- Sink: una o varias transferencias de curl.
- Primitiva: añadir un segundo operando HTTP controlado.
- Payload mínimo: `https://allowed.lab/a http://listener.lab/token`.
- Significado de cada componente: primer operando satisface la función; segundo solicita el listener.
- Resultado esperado: dos solicitudes dentro del mismo proceso.
- Control negativo: el mismo texto pasado como un único `argv[1]`.
- Restricción observada: esquemas locales están bloqueados.
- Por qué falla la variante básica: `;` `id` no es opción ni operando válido para una shell ausente.
- Hipótesis de adaptación: abusar de operandos soportados por curl.
- Payload adaptado: segundo URL HTTP del laboratorio.
- Por qué debería funcionar: el wrapper lo convierte en otro elemento y curl admite múltiples URL.
- Evidencia: `argv`, dos callbacks y un solo PID de curl.
- Cuándo no funcionaría: un solo elemento, allowlist por cada URL o wrapper que inserta -- y limita operandos.

Variaciones según el contexto

Construcción	Espacio	;	Riesgo principal
[programa, entrada]	parte del mismo argumento	literal	operando controlado
[programa, *split(entrada)]	crea argumentos	literal	option/argument injection
shell=True con cadena	shell splitting	operador	command injection y argumentos
glob de shell *	nombres se vuelven argumentos	depende del nombre	option injection por filenames

Filtros y bypasses

Bloquear `;` no corrige inyección de opciones. La defensa debe controlar la construcción de `argv`, validar cada operando, insertar -- cuando la herramienta lo soporte y evitar parsers de texto intermedios.

Evidencias de confirmación

`argv` adicional o efecto documentado de una opción/operando en el mismo proceso. Una firma de salida parecida a curl solo identifica una hipótesis de herramienta.

Escalado de impacto

Estudiar únicamente capacidades documentadas de la versión, comenzar con opciones de diagnóstico y demostrar el impacto mínimo. No asumir que todas las herramientas o compilaciones soportan los mismos protocolos/hooks.

Errores frecuentes

- Afirmar que un espacio siempre divide.
- Confundir segundo URL de curl con segundo comando.
- Usar opciones de otra versión.
- Suponer que -- es universal o que el wrapper lo coloca correctamente.
- Saltar de error de opción a ejecución arbitraria.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
espacio aparece escapado en un error	un único argumento	log argv y longitud
; id literal, opción cambia salida	parser de opciones sin shell	comparar PID y argv
opción desconocida	versión/herramienta distinta	obtener versión y manual oficial
-- detiene la opción	parser implementa fin de opciones	comparar antes/después de --
segundo URL no se solicita	un solo argumento o política por URL	log argv y listener por operando
aparecen dos procesos	shell/wrapper adicional	árbol de procesos y comando final

Mitigaciones

Construir argv explícito, un elemento por valor, evitar split de entrada, usar -- cuando esté documentado, allowlist de opciones y operandos, API nativa y privilegio mínimo.

Relación con pentesting y certificaciones

Se evalúa la capacidad de dibujar argv y atribuir el efecto al parser correcto.

Caso guiado

Caso A — Básico: dos recursos en un importador

El log muestra argv=["curl","https://allowed/a","http://listener/t1"]; llegan dos peticiones y existe un solo PID.

Observación: la entrada creó dos operandos. Qué sé: el wrapper divide y curl procesa ambas URL. Hipótesis: argument injection o shell. Experimento: incluir ; como token y revisar árbol de procesos. Resultado: ; es un operando inválido y no nace otro proceso. Conclusión: inyección de argumentos, no de comandos. Siguiente paso: revisar capacidades documentadas y política por operando.

Caso de adaptación

Caso B — El espacio permanece dentro del argumento

El mismo payload en otra versión produce argv=["curl","https://allowed/a http://listener/t1"] y URL rejected: Malformed input.

Observación: no hay segundo elemento. Qué sé: un espacio no fue dividido. Hipótesis: lista directa o quoting del wrapper. Experimento: log de longitud y opción inocua. Resultado: siempre dos elementos totales: programa y valor. Conclusión: la variante básica falla porque no existe splitting; no hay adaptación legítima con otro separador sin nueva

evidencia. Siguiendo paso: descartar esta vía.

Caso C — Transferencia: perfil de conversión

Un conversor recibe `profile="-loglevel debug"`; el log muestra que el wrapper aplica `shlex.split(profile)` y la herramienta cambia verbosidad. No se revela la técnica en el título.

Resolución: texto -> split -> opciones de la herramienta. El control pasa el mismo texto como un elemento y recibe "unrecognized option". La primitiva transferida es controlar opciones, no ejecutar una shell.

Caso D — Falso positivo: medidor de progreso

La respuesta contiene % Total y % Received, pero ninguna opción inocua altera el comportamiento y el backend usa una biblioteca que copia ese formato.

Observación: firma compatible con curl. Hipótesis: CLI curl o biblioteca. Experimento: árbol de procesos, error de opción y versión. Resultado: no hay proceso curl ni parser de opciones. Conclusión: la firma no confirma argument injection.

Comparación: Command Injection vs Argument Injection

Experimento	Shell vulnerable	argv con splitting	argv único
<code>; printf MARK</code>	ejecuta MARK	<code>;/printf</code> son argumentos	texto literal único
opción inocua documentada	puede llegar tras splitting	cambia herramienta	queda dentro del operando
árbol de procesos	puede mostrar shell y segundo proceso	un proceso objetivo	un proceso objetivo
log argv	resultado tras expansión	varios elementos controlados	un elemento controlado

Ejercicios

1. Dibuja argv para las tres llamadas Python de Fundamentos.
2. Explica por qué dos callbacks de curl no implican dos comandos.
3. Diseña un control con `--` sin asumir que la herramienta lo soporta.
4. Distingue option injection de un operando adicional.

Resumen

La inyección de argumentos depende de quién crea argv. Un espacio solo divide cuando una capa lo interpreta; el efecto pertenece al parser de opciones del programa, no a una shell por defecto.

Chuleta operativa

1. Código o log de construcción.
2. Número y contenido de argv.
3. Shell sí/no.
4. Opción inocua y versión.
5. `--` documentado.
6. PID y efecto mínimo.

Referencias

- [Python subprocess](#)
- [curl manual](#)

- [GNU Bash Reference Manual](#)
- [Concepto interno](#) (argument-injection)

Navegación

Anterior: [Command injection](#). Índice: [curso](#). Práctica: [cuaderno](#).

Autenticación, autorización y sesiones en APIs

Objetivos de aprendizaje

Modelar estados de identidad, distinguir autenticación de autorización, verificar rotación, expiración y revocación, y diseñar controles cruzados entre sesiones, objetos y acciones.

Prerrequisitos

HTTP, cookies, bearer tokens, estados, controles negativos y MFA.

Fundamentos técnicos

Autenticar establece una identidad; la sesión relaciona solicitudes con ese estado; autorizar decide si esa identidad puede realizar una acción sobre un objeto. Un token válido puede estar correctamente autenticado y aun así acceder a recursos indebidos por un fallo de autorización.

Modelo mental

```
anónimo -> credencial primaria -> preautenticado -> MFA -> autenticado
      -> elevación/acción sensible -> logout/expiración/revocación
      |
      +--> autorización por identidad + acción + objeto
```

Superficie de ataque

Login, refresh, logout, recuperación, cambio de contraseña, “recordar dispositivo”, endpoints de objetos y cambios de rol.

Cómo identificarla

- La sesión rota o no rota tras cambiar privilegio.
- Token viejo sigue o deja de funcionar tras refresh/logout.
- Mismo objeto responde distinto para dueño, otro usuario y anónimo.
- Timeouts se aplican en servidor, no solo en interfaz.

Preguntas que debo hacerme

1. ¿Qué material autentica cada petición?
2. ¿Qué estado representa y dónde se conserva?
3. ¿Cuándo se rota o revoca?
4. ¿Qué autorización se aplica por objeto y acción?
5. ¿Hay caché o gateway que explique la respuesta?

Prueba mínima

Construir una matriz con sesión A, sesión B y anónimo sobre recurso A y B, usando lectura y una acción no destructiva. Después probar token anterior/nuevo en refresh y logout.

Construcción progresiva del payload

En esta técnica el “payload” suele ser una secuencia: obtener sesión, capturar identificador, cambiar una sola precondition, repetir exactamente el recurso y comparar estado server-side. No se manipula un token opaco al azar.

Anatomía de los payloads

- Contexto de entrada: Authorization: Bearer <token> y path /orders/1042.

- Sintaxis original: token identifica sesión; path identifica objeto.
- Entrada controlada: token y objeto solicitado.
- Transformaciones conocidas: gateway valida firma; API resuelve sesión.
- Parser final: middleware de autenticación y política de autorización.
- Sink: lectura de pedido.
- Primitiva: acceder con sesión B a objeto A.
- Payload mínimo: petición válida de B cambiando solo 1043 por 1042.
- Significado de cada componente: el token conserva la identidad B; el path selecciona el objeto atribuido a A.
- Resultado esperado: 403 o respuesta indistinguible sin datos en una implementación segura.
- Control negativo: B->B, A->A y anónimo->A.
- Restricción observada: ambos usuarios reciben 200 para 1042.
- Por qué falla la variante básica: quitar token solo prueba autenticación.
- Hipótesis de adaptación: mantener identidad válida y cambiar objeto.
- Payload adaptado: matriz identidad-objeto.
- Por qué debería funcionar: separa autenticación de pertenencia.
- Evidencia: B recibe datos exclusivos de A reproduciblemente.
- Cuándo no funcionaría: autorización correcta o recurso público compartido.

Variaciones según el contexto

Sesión server-side, token autosuficiente y refresh token tienen ciclos distintos. Expiración idle, absoluta y de credencial no son equivalentes. Logout efectivo requiere invalidación server-side cuando el mecanismo lo permite; borrar una cookie local no basta para demostrar revocación.

Filtros y bypasses

No trates 401/403 como oráculos perfectos: gateways y aplicaciones los usan de forma inconsistente. Mantén cuerpo, método, objeto y sesión controlados; registra redirecciones y caché.

Evidencias de confirmación

Acceso indebido a objeto/acción, aceptación de una sesión que debería estar revocada o ausencia de rotación con impacto demostrable. Una cookie sin HttpOnly es una debilidad distinta, no bypass por sí sola.

Escalado de impacto

Ampliar de lectura a acciones solo si es necesario y seguro; probar cambio de contraseña, elevación o revocación con cuentas del laboratorio. Evitar tocar datos de otros usuarios reales.

Errores frecuentes

- Probar sin token y concluir autorización segura.
- Confundir endpoint público con bypass.
- No fijar caché o sesión.
- Asumir que expiración del JWT revoca refresh tokens.
- Tratar ocultación del botón como control server-side.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
sin token y con token responden igual	recurso público o token ignorado	/me y recurso privado conocido
logout borra cookie, token aún funciona	revocación solo cliente	reutilizar token desde cliente separado
token viejo y nuevo funcionan	ventana de rotación o fallo	medir duración y evento de revocación
B lee objeto A	autorización por objeto ausente	matriz B->B/A y acción distinta
200 con cuerpo vacío	autorización en lógica o caché	comparar datos, cabeceras y backend
sesión cambia tras login	rotación esperada	comprobar que la anterior se invalida

Mitigaciones

Estado server-side coherente, rotación tras cambios de privilegio, expiración idle/absoluta, revocación, cookies seguras, autorización por petición y objeto, y pruebas automatizadas de matrices de acceso.

Relación con pentesting y certificaciones

Se evalúa el modelo de estado y la matriz de controles, no la cantidad de tokens manipulados.

Caso guiado

Caso A — Básico: pertenencia de pedidos

Cuenta B cambia /orders/1043 por /orders/1042 y recibe los datos de A.

Observación: B está autenticada y lee A. Qué sé: autenticación funciona; pertenencia no. Hipótesis: autorización ausente o caché. Experimento: marcadores anti-caché, varios objetos y acción de lectura. Resultado: patrón sigue el ID, no la caché. Conclusión: bypass de autorización por objeto. Siguiendo paso: demostrar alcance mínimo.

Caso de adaptación

Caso B — Logout aparente

La UI vuelve al login tras logout, pero una copia del bearer token sigue accediendo a /api/me.

Observación: interfaz cerrada, token válido. Hipótesis: token no revocable hasta expirar o endpoint de logout incompleto. Experimento: cliente separado, token de acceso y refresh por separado, tiempos. Resultado: access token sigue 15 min y refresh queda revocado. Conclusión: comportamiento documentado del diseño, no necesariamente bypass; el riesgo depende del requisito de revocación inmediata. Siguiendo paso: comparar con especificación del sistema.

Caso C — Transferencia: cambio de correo

Una acción sensible acepta la sesión autenticada hace horas sin reautenticación. No se nombra el fallo en el título.

Resolución: sesión válida -> acción de alto riesgo -> ausencia de verificación reciente. El experimento compara sesión recién autenticada y antigua, no manipula el correo. Se documenta como política de reautenticación débil si el requisito la exige.

Caso D — Falso positivo: endpoint de catálogo

/api/plans responde igual sin token y con token. El alumno sospecha token ignorado.

Experimento: /api/me y /api/billing con las mismas condiciones. Resultado: catálogo es público; privados devuelven 401. Conclusión: la igualdad no demuestra bypass.

Ejercicios

1. Construye una matriz de dos usuarios, dos objetos y dos acciones.
2. Distingue expiración de access token y revocación de refresh token.
3. Diseña un control para caché en una prueba de autorización.
4. Explica cuándo 401 frente a 403 no cambia la conclusión.

Resumen

Autenticación, sesión y autorización son capas distintas. Las pruebas útiles cambian una identidad, objeto, acción o estado cada vez y verifican el ciclo completo.

Chuleta operativa

1. Estado e identidad.
2. Token/cookie y ciclo.
3. Matriz identidad-objeto-acción.
4. Rotación y token anterior.
5. Idle/absoluta/revocación.
6. Caché y controles.

Referencias

- [OWASP Authentication Cheat Sheet](#)
- [OWASP Session Management Cheat Sheet](#)
- [PortSwigger: access control](#)
- [Concepto interno](#) (auth-session-security)

Navegación

Anterior: [MFA/OTP](#). Índice: [curso](#). Práctica: [cuaderno](#).

Controles client-side y validacion del servidor

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

JavaScript, campos hidden, botones deshabilitados, validaciones HTML y datos almacenados en el navegador mejoran la interfaz, pero el cliente controla la petición final. El servidor debe repetir validación, autenticación y autorización sobre cada operación.

Burp o curl permiten omitir la interfaz y cambiar parámetros, métodos, cookies y cuerpos. Si el backend confía en `role=admin`, `price`, `user_id` o una comprobación hecha solo en JavaScript, el control se puede saltar.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la interfaz oculta acciones, limita valores o decide permisos antes de enviar una petición al backend.

Cómo identificarla

- hidden
- disabled
- localStorage
- role
- price
- user_id
- Validación JavaScript

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Captura una petición válida, cambia una sola propiedad controlada por el cliente y compara la respuesta con otra cuenta o rol.

Evidencia esperada: Evidencia reproducible de que el servidor acepta o rechaza correctamente el cambio, separada del comportamiento visual del navegador.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -i "$URL/profile?user_id=1001" -H "Cookie: session=$COOKIE"
```

Objetivo y contexto: Reproduce directamente una petición autenticada para comprobar si el servidor valida la propiedad del recurso.

Resultado esperado: Un 403 confirma control; datos de otra cuenta confirman un fallo de autorización.

Anatomía de los payloads

La primera prueba es `curl -i "$URL/profile?user_id=1001" -H "Cookie: session=$COOKIE"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Evidencia reproducible de que el servidor acepta o rechaza correctamente el cambio, separada del comportamiento visual del navegador.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Observa la petición real.
- Identifica el dato que impone la UI.
- Repite en Repeater.
- Cambia solo ese dato.
- Compara por cuenta, status y cuerpo.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Cambiar la interfaz sin conseguir una respuesta distinta del servidor no demuestra impacto.
- Un 403 consistente al repetir la petición indica que el backend si aplica el control.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Todo valor controlado por el navegador puede modificarse; la seguridad real debe imponerse en el servidor. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(client-side-controls\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Inyección de comandos de sistema operativo

Objetivos de aprendizaje

Distinguir shell de ejecución directa, predecir el significado de metacaracteres, construir pruebas in-band y blind con controles y adaptar quoting o canal sin probar separadores al azar.

Prerrequisitos

Procesos, argv, Bash, quoting, JSON, redirecciones y medición temporal.

Fundamentos técnicos

Command injection requiere que entrada controlada altere una orden interpretada por una shell u otro intérprete de comandos. En ejecución directa, el kernel recibe un ejecutable y una lista de argumentos: `;`, `&&` o `|` no tienen gramática propia y pueden llegar como texto al programa.

```
sh -c "ping -c 1 <entrada>"      -> la shell analiza operadores
execve("ping", ["ping", "-c", "1", entrada]) -> ping recibe un argumento
```

Modelo mental

```
HTTP/JSON -> valor -> concatenación/quoting -> ¿shell? -> expansiones/redirecciones
\----> ¿argv? -> parser de opciones
```

Superficie de ataque

Diagnósticos de red, conversión de ficheros, backups, búsquedas, integraciones y tareas que invocan utilidades. La presencia de un proceso externo no implica shell.

Cómo identificarla

- Un marcador de segundo comando aparece fuera de la salida normal.
- Una demora proporcional se reproduce frente a controles intercalados.
- Un callback OOB contiene token único generado por la orden.
- Los errores cambian según quoting u operador de una shell identificada.

Preguntas que debo hacerme

1. ¿Existe shell o ejecución directa?
2. ¿Dentro de qué comillas se inserta el valor?
3. ¿Qué operadores entiende esa shell?
4. ¿Se captura stdout, stderr o ningún canal?
5. ¿Qué identidad, entorno y directorio tiene el proceso?

Prueba mínima

Con salida visible, producir un marcador fijo mediante una orden inocua. Sin salida, usar una demora corta y dos duraciones con varias muestras, o un callback al listener autorizado. Siempre mantener petición original y una cadena literal como controles.

Construcción progresiva del payload

Para `sh -c "ping -c 1 $host"`: conservar un host válido, determinar quoting, añadir un separador, emitir MARK y predecir dónde aparece. Para `execve`, esa misma cadena no es un payload de command injection: es un único operando de ping.

Anatomía de los payloads

- Contexto de entrada: JSON host.
- Sintaxis original: `sh -c "ping -c 1 <host>"`.
- Entrada controlada: final de la orden, sin quoting adicional.
- Transformaciones conocidas: JSON decode y concatenación.
- Parser final: `/bin/sh` confirmado.
- Sink: ejecución de orden.
- Primitiva: segundo comando que emite marcador.
- Payload mínimo: `127.0.0.1; printf CI_OK`.
- Significado de cada componente: host válido; `;` termina la primera orden; `printf` emite marcador.
- Resultado esperado: salida de ping seguida de `CI_OK`.
- Control negativo: `127.0.0.1; printf CI_NO` enviado a una ruta que usa `execve`.
- Restricción observada: `stdout` no se devuelve en producción del laboratorio.
- Por qué falla la variante básica: la orden se ejecuta, pero el canal no es visible.
- Hipótesis de adaptación: usar efecto temporal proporcional.
- Payload adaptado: separador más una demora corta permitida en el laboratorio.
- Por qué debería funcionar: la shell espera al segundo proceso.
- Evidencia: distribución separada y proporcional para 2 s y 4 s.
- Cuándo no funcionaría: ejecución asíncrona, timeout, shell ausente o utilidad de demora bloqueada.

Variaciones según el contexto

`;` `id` puede: ejecutarse si una shell lo analiza sin quoting protector; aparecer literalmente si es un argumento; causar error si el programa valida el host completo; o no producir señal si la entrada se ignora o la salida no se captura.

Filtros y bypasses

Un carácter bloqueado no justifica probar todos los demás. Determina si el rechazo ocurre en JSON, validación, shell o programa. Una adaptación cambia quoting, operador o canal solo para probar una hipótesis concreta.

Evidencias de confirmación

Marcador, demora proporcional o callback atribuible a una segunda orden. Un error distinto al incluir `;` solo prueba que el carácter afectó alguna capa.

Escalado de impacto

Confirmar identidad y entorno con una capacidad mínima, evaluar lectura/escritura estrictamente necesaria y evitar saltar a una reverse shell si el canal existente basta para demostrar impacto.

Errores frecuentes

- Agrupar APIs con y sin shell como equivalentes.
- Confundir error de ping con error de shell.
- Usar una sola medida temporal.
- Escapar para la shell local, no para la remota.
- Interpretar fallo de reverse shell como refutación de ejecución.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
; id aparece en error de host	argv directo o quoting	observar argv; comparar expansión shell
; id cambia error, sin salida	shell parcial o validación	marcador temporal/OOB y stderr separado
demora consistente y proporcional	ejecución blind	alternar controles y dos duraciones
id funciona, conexión saliente no	egress/DNS/entorno	callback DNS/HTTP controlado y variables de red
; bloqueado antes del proceso	filtro textual	localizar fase; no enumerar separadores
JSON devuelve 400	transporte inválido	serializar objeto y verificar bytes

Mitigaciones

Usar APIs nativas o ejecución directa con lista fija de argumentos, validar valores semánticamente, eliminar shell, fijar entorno y privilegios, y registrar la invocación sin datos sensibles.

Relación con pentesting y certificaciones

La destreza evaluada es demostrar la gramática que procesa la entrada y elegir un canal robusto.

Caso guiado

Caso A — Básico: diagnóstico de host

La respuesta a 127.0.0.1; printf MARK contiene salida de ping y MARK.

Observación: aparece un marcador de una segunda orden. Qué sé: alguna capa interpretó ;. Hipótesis: shell en backend o interpretación inesperada anterior. Experimento: control literal, captura de comando del laboratorio y operador condicionado. Resultado: log sh -c y marcador solo con separador. Conclusión: command injection in-band. Siguiente paso: registrar UID y limitar impacto.

Caso de adaptación

Caso B — Sin salida visible

El mismo endpoint devuelve siempre 202 job queued.

Observación: no hay canal in-band. Hipótesis: orden asíncrona, entrada no alcanzada o salida descartada. Experimento: demoras 2/4 s no sirven si la respuesta es asíncrona; se usa callback con token desde el worker del laboratorio y un control sin segundo comando. Resultado: el callback llega desde el worker solo con payload. Conclusión: ejecución blind confirmada; la ausencia de demora no la refutaba. Siguiente paso: documentar cola e identidad.

Caso C — Transferencia: nombre del archivo de backup

Una tarea web crea tar con un nombre de archivo proporcionado. El alumno observa que un nombre con ; printf MARK genera un fichero adicional MARK en el directorio de pruebas.

Resolución: nombre -> cadena de shell -> tar -> segundo comando. Se confirma con un nombre literal escapado como control y con el comando construido. La función no se llamaba "ejecutar comando", pero la primitiva es idéntica.

Caso D — Falso positivo: ping recibe todo

127.0.0.1; id devuelve ping: 127.0.0.1; id: Name or service not known.

Observación: el texto llegó a ping. Hipótesis: shell protegida o execve. Experimento: traza argv y expansión \$(printf X). Resultado: un único argumento literal. Conclusión: command injection descartada; revisar argument/option injection solo si puede crearse otro argumento.

Ejercicios

1. Predice el resultado de la misma entrada bajo sh -c y execve.
2. Diseña una medición temporal resistente a ruido.
3. Explica por qué una reverse shell fallida no refuta printf MARK exitoso.
4. Serializa por capas una cadena JSON que contenga comillas de shell.

Resumen

Los operadores solo son operadores para una shell. Primero se identifica parser y quoting; después se construye una primitiva y un canal causal.

Chuleta operativa

1. Cadena final o argv.
2. Shell y quoting.
3. Marcador mínimo.
4. Stdout, stderr, tiempo u OOB.
5. Control literal y repetición.
6. UID, entorno y directorio.

Referencias

- [PortSwigger: OS command injection](#)
- [GNU Bash Reference Manual](#)
- [Python subprocess](#)
- [Concepto interno](#) (command-injection)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).

Enumeracion web con respuestas comodin (wildcard)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Algunas apps devuelven una pagina generica (200 OK) para toda ruta que no existe, en vez de un 404. Eso hace que gobuster/ffuf marquen como 'validas' todas las rutas y te llenen de falsos positivos. La clave es que las respuestas comodin comparten un tamano/estructura fijos, asi que filtras por ese tamano en lugar de por el codigo.

Un front controller o un handler catch-all sirve la misma plantilla (login, home, error bonito) para rutas desconocidas y responde 200. El fuzzer solo mira el status por defecto, asi que no ve la diferencia hasta que le dices que ignore ese tamano concreto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un escaneo de directorios devuelve casi todo en 200 con longitudes identicas, o cuando abrir una ruta inventada a mano te da la misma pagina que una ruta real. Sospecha en apps con enrutado propio (PHP con .htaccess, SPA, frameworks).

Cómo identificarla

- Casi todas las lineas del escaneo son 200 y comparten la misma longitud.
- Una ruta inventada a mano devuelve la misma pagina que el sitio real.
- Tras encontrar login.php, buscar backups con '-x bak' sobre la palabra 'login' prueba login.bak pero no login.php.bak: repite el fuzzing sobre los nombres COMPLETOS ya descubiertos (login.php.bak/.old/.save).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pide una ruta imposible (p.ej. /qwerty-noexiste-123) y anota su status y bytes; si es 200 con un tamano fijo, filtra ese tamano en el fuzzer.

Evidencia esperada: El escaneo pasa de cientos de falsos 200 a una lista corta de rutas realmente distintas.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
gobuster dir -u $URL/ -w /usr/share/wordlists/dirb/common.txt -x php,bak,old,txt --exclude-length 1491
```

Objetivo y contexto: `--exclude-length` descarta las respuestas comodín de ese tamaño fijo (aquí 1491 bytes), dejando solo lo que difiere de verdad. Sustituye 1491 por el tamaño que midas.

Resultado esperado: Una lista corta de rutas con longitud distinta a la comodín. Ahí están los ficheros reales (login.php, api_login.php, etc.).

Capa 2: prueba documentada

```
ffuf -u $URL/login.phpFUZZ -w <(printf '%s\n' .bak .old .save .swp ~) -mc all -fs 1491
```

Objetivo y contexto: Segunda pasada sobre un fichero YA descubierto para encontrar copias por extensión del nombre completo, el fallo que `'-x bak'` sobre la palabra base no cubre.

Resultado esperado: login.php.bak/.old/.save si existen. Un backup servido como texto = código fuente expuesto con credenciales y endpoints.

Anatomía de los payloads

La primera prueba es `gobuster dir -u $URL/ -w /usr/share/wordlists/dirb/common.txt -x php,bak,old,txt --exclude-length 1491`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El escaneo pasa de cientos de falsos 200 a una lista corta de rutas realmente distintas.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Mide la respuesta comodín pidiendo una ruta imposible y apunta bytes/palabras.
- Relanza el fuzzer excluyendo ese tamaño para eliminar los falsos positivos.
- Sobre los ficheros reales descubiertos, haz una segunda pasada buscando copias por extensión del nombre completo (.php.bak, .php.old, .php.save, .swp).

- Abre los backups: se sirven como texto y suelen filtrar credenciales, endpoints reales y logica (fuente expuesta).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor devuelve 404 real para rutas inexistentes (el filtrado por status normal ya sirve).
- Las respuestas comodín tienen longitud variable (contenido dinámico): filtra entonces por regex/palabras, no por bytes fijos.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El servidor responde 200 a CUALQUIER ruta, así que el código HTTP no distingue lo real de lo inexistente: filtra por longitud o palabras, no por status. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (enum-wildcard-responses)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Seguridad de subida de ficheros

Objetivos de aprendizaje

Separar aceptación, almacenamiento, recuperación, interpretación y ejecución; identificar qué metadato o contenido decide cada capa y confirmar una capacidad cada vez.

Prerrequisitos

Multipart, MIME, rutas, servidores estáticos, handlers, autorización y parsers de formatos.

Fundamentos técnicos

Una subida correcta solo demuestra que la aplicación aceptó bytes. El impacto depende de pasos posteriores:

```
subida -> validación -> nombre/ubicación -> almacenamiento -> recuperación
      -> cabeceras/handler -> interpretación -> posible ejecución
```

El Content-Type de la parte, el nombre y la extensión son datos del cliente; la defensa puede usarlos como una señal, pero debe validarlos junto al contenido y aislar el almacenamiento. Ningún campo aislado decide universalmente si un fichero se ejecuta.

Modelo mental

Para cada prueba registra: nombre declarado, MIME declarado, firma/contenido, nombre final, ubicación, URL, cabeceras de descarga y componente que procesa los bytes.

Superficie de ataque

Avatares, adjuntos, importaciones, temas, documentos, archivos comprimidos y endpoints PUT. Cada flujo puede almacenar en disco local, objeto remoto o cola de procesamiento.

Cómo identificarla

- La respuesta entrega un identificador o nombre final.
- El fichero puede recuperarse con identidad y cabeceras conocidas.
- El contenido se sirve estático, se descarga como adjunto o pasa por un parser.
- Solo una respuesta del handler o un marcador interpretado confirma ejecución server-side.

Preguntas que debo hacerme

1. ¿Qué controles se aplican y en qué orden?
2. ¿Se renombra y dónde se almacena?
3. ¿Quién puede recuperar el objeto?
4. ¿Qué Content-Type y Content-Disposition devuelve el servidor?
5. ¿Qué handler, si alguno, interpreta la extensión o bytes?

Prueba mínima

Subir un fichero de texto con token único, recuperarlo mediante el flujo normal y comparar hash, nombre y cabeceras. Esa prueba no intenta ejecución.

Construcción progresiva del payload

1. Texto marcador permitido.
2. Mismo contenido con un metadato cambiado.
3. Mismo nombre con contenido inválido para el tipo.

4. Localizar nombre final y política de acceso.
5. Comparar descarga forzada y renderizado.
6. Solo en un laboratorio preparado, usar un marcador interpretado inocuo para comprobar handler.

Anatomía de los payloads

- Contexto de entrada: parte multipart file.
- Sintaxis original: nombre probe.txt, tipo text/plain, bytes UPLOAD-7F31.
- Entrada controlada: metadatos y bytes de la parte.
- Transformaciones conocidas: parser multipart, renombrado UUID.
- Parser final: almacenamiento de objetos; sin intérprete.
- Sink: escritura y posterior descarga.
- Primitiva: persistir y recuperar bytes idénticos.
- Payload mínimo: fichero marcador de texto.
- Significado de cada componente: token único permite atribución y hash.
- Resultado esperado: 201, ID y descarga idéntica.
- Control negativo: ID inexistente y acceso de otra cuenta.
- Restricción observada: extensiones no permitidas se renombran .bin.
- Por qué falla la variante básica: cambiar solo Content-Type no altera nombre final ni handler.
- Hipótesis de adaptación: estudiar la capa que realmente decide interpretación.
- Payload adaptado: no procede si el objeto se sirve con descarga forzada y sin handler.
- Por qué debería funcionar: solo habría ejecución si una ruta posterior interpretase los bytes.
- Evidencia: cabeceras application/octet-stream, attachment y ausencia de handler.
- Cuándo no funcionaría: procesamiento posterior, servidor mal configurado o ubicación ejecutable.

Variaciones según el contexto

Un SVG puede ser seguro como descarga y peligroso si se renderiza same-origin. Un ZIP puede ser seguro almacenado y peligroso al extraerse sin validar rutas. Una imagen puede activar vulnerabilidades del parser aun sin ejecución por extensión.

Filtros y bypasses

Cambiar extensión, MIME y bytes a la vez impide saber qué control falló. Aísla cada variable. Una allowlist de extensión es una capa; no sustituye renombrado, aislamiento, límites, autorización y desactivación de handlers.

Evidencias de confirmación

Aceptación, persistencia, recuperación e interpretación requieren evidencias separadas. Un 201 no confirma ubicación pública; una URL pública no confirma ejecución; un 200 puede ser descarga estática.

Escalado de impacto

Evaluar acceso entre usuarios, overwrite, path traversal en nombre, serving same-origin, extracción y parsers. Solo probar ejecución server-side en entorno preparado y con marcador no destructivo.

Errores frecuentes

- Titular la técnica “subida a shell”.
- Confiar en Content-Type enviado por el cliente.

- Probar doble extensión sin saber qué componente selecciona handler.
- Buscar rutas por fuerza bruta cuando existe un ID.
- Confundir contenido PHP servido como texto con ejecución.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
201, pero URL original da 404	renombrado o almacenamiento externo	usar ID devuelto y endpoint de descarga
bytes cambian	transcodificación/CDR	comparar hash, dimensiones y metadatos
extensión ejecutable se descarga	sin handler o attachment	marcador interpretado seguro y cabeceras
MIME falso se rechaza	inspección de contenido	cambiar solo bytes con extensión permitida
otra cuenta recibe 403	autorización por objeto	matriz identidad-ID-acción
fichero desaparece	cuarentena, TTL o antivirus	observar estado asíncrono y tiempos

Mitigaciones

Allowlist necesaria, validación de firma/contenido, nombre generado, límites, autorización, almacenamiento fuera del webroot o en otro host, serving mediante handler seguro, nosniff, descarga cuando proceda y análisis/CDR según riesgo.

Relación con pentesting y certificaciones

Se evalúa el pipeline demostrado, no si se obtuvo una shell.

Caso guiado

Caso A — Básico: adjunto recuperable

Un .txt con token recibe ID 8c2; GET /files/8c2 devuelve bytes idénticos con Content-Disposition: attachment.

Observación: aceptación, almacenamiento y recuperación confirmados. Qué sé: no hay interpretación. Hipótesis: objeto privado o público. Experimento: sin sesión y con otra cuenta. Resultado: ambos 403. Conclusión: flujo básico autorizado; no existe vulnerabilidad demostrada. Siguiendo paso: revisar controles de tipo y tamaño por separado.

Caso de adaptación

Caso B — El nombre se conserva, pero no se ejecuta

El laboratorio acepta probe.phtml y permite descargarlo. El cuerpo PHP aparece literal.

Observación: extensión conservada y contenido recuperable. Hipótesis: handler PHP o serving estático. Experimento: marcador que, si se interpretara, produciría texto distinto; revisar cabeceras y configuración de ruta. Resultado: bytes literales, application/octet-stream, host de objetos separado. Conclusión: no hay ejecución; adaptar extensión no cambia el componente. Siguiendo paso: documentar almacenamiento potencialmente riesgoso solo según consumidores posteriores.

Caso C — Transferencia: paquete de traducción

Una función acepta ZIP y lo extrae. Un archivo con ruta ../marker.txt aparece fuera del directorio de traducciones del laboratorio.

Resolución: upload -> almacenamiento temporal -> parser ZIP -> normalización de rutas -> escritura. La primitiva transferida no es ejecución: es path traversal durante extracción. Se confirma con ruta interna normal y marcador fuera de base.

Caso D — Falso positivo: respuesta “uploaded”

El frontend muestra “uploaded”, pero la respuesta HTTP es 422 y no existe ID; el mensaje se genera antes de esperar al servidor.

Observación: éxito visual. Hipótesis: persistencia o UI optimista. Experimento: respuesta cruda, recarga y listado. Resultado: no hay objeto. Conclusión: ni siquiera se confirmó subida.

Ejercicios

1. Diseña una matriz que cambie solo extensión, MIME o bytes.
2. Propón evidencia independiente para cada flecha del pipeline.
3. Distingue serving estático, descarga y handler con cabeceras y marcador.
4. Analiza un ZIP sin asumir que el riesgo principal es ejecución.

Resumen

Subir no implica almacenar, recuperar, interpretar ni ejecutar. El análisis seguro confirma cada transición y atribuye el comportamiento al componente correcto.

Chuleta operativa

1. Nombre, MIME, bytes y tamaño.
2. ID/nombre final y ubicación.
3. Hash recuperado.
4. Autorización.
5. Cabeceras de serving.
6. Parser/handler.
7. Una variable por prueba.

Referencias

- [PortSwigger: file upload](#)
- [OWASP File Upload Cheat Sheet](#)
- [Concepto interno](#) (file-upload)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).

Seguridad de GraphQL

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

GraphQL concentra muchos recursos en un endpoint y permite seleccionar campos mediante un schema.

Introspection abierta, resolveres sin autorización y consultas costosas amplían la superficie de ataque.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando observas `/graphql`, `content-type application/graphql` o cuerpos JSON con query.

Cómo identificarla

- `__schema`
- `query`
- `mutation`
- `errors`
- `data`

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Una petición válida devuelve datos o errores GraphQL estructurados.

Evidencia esperada: Tienes el schema disponible o una lista de operaciones y sabes donde probar permisos y límites.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.

5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/graphql" -H 'Content-Type: application/json' --data '{"query":"{ __typename }"}'
```

Objetivo y contexto: Confirma que el endpoint procesa GraphQL.

Resultado esperado: Respuesta data o error GraphQL.

Capa 2: prueba documentada

```
curl -sS "$URL/graphql" -H 'Content-Type: application/json' --data @introspection.json
```

Objetivo y contexto: Solicita el schema si esta permitido.

Resultado esperado: Tipos, campos y operaciones disponibles.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/graphql" -H 'Content-Type: application/json' --data '{"query":"{ __typename }"}'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tienes el schema disponible o una lista de operaciones y sabes donde probar permisos y límites.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba introspection.
- Enumera queries y mutations.
- Repite operaciones con identidades distintas.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No aplica a APIs REST sin endpoint GraphQL identificable.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Enumerar schema, queries, mutations y controles de autorizacion en GraphQL. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (graphql-security)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

IDOR / Broken Object Level Authorization (BOLA)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un IDOR (Insecure Direct Object Reference) ocurre cuando la app usa un identificador (un ID numerico, un UUID, un email) para acceder a un recurso, y el backend confía en que 'si conoces el ID, tienes permiso'. Si cambias el ID por el de otro usuario y el backend no revalida que ESE ID te pertenece, accedes a datos o acciones ajenas. BOLA es el mismo bug con nombre mas moderno, centrado en APIs.

El programador comprueba que ESTAS autenticado (tienes un token valido) pero se olvida de comprobar que ese token tiene permiso sobre ESE recurso concreto. Autenticacion (quien eres) y autorizacion (que puedes tocar) son cosas distintas, y este bug es exactamente confundirlas.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier endpoint que reciba un identificador de recurso: /api/users/{id}, /api/orders/{id}, /invoice/{id}.pdf, ?user_id=. Sospecha mas si los IDs son secuenciales (facil de adivinar) que si son UUIDs (aunque UUIDs filtrados en otra respuesta tambien sirven).

Cómo identificarla

- IDs numericos secuenciales en la URL o el JSON de respuesta.
- El frontend oculta botones/vistas segun el rol, pero la llamada a la API subyacente no cambia.
- Cambiar el ID devuelve 200 con datos que no son tuyos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Con tu sesion, pide el mismo endpoint cambiando el ID por uno que sepas que no es tuyo (otro usuario, otro pedido).

Evidencia esperada: Un 200 con datos de otro usuario/recurso (o una accion aceptada sobre un recurso ajeno) confirma el IDOR. Un 403/404 descarta ese endpoint concreto.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/api/users/1001" -H "Authorization: Bearer $TOKEN"
```

Objetivo y contexto: Pide un recurso con un ID que probablemente no es el tuyo, usando tu propio token. Si el backend solo mira 'hay token valido' y no 'este token puede ver el 1001', el IDOR se confirma.

Resultado esperado: Datos de un usuario que no eres tu (nombre, email, direccion) devueltos con 200. Repite con varios IDs cercanos para confirmar el patron.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/api/users/1001" -H "Authorization: Bearer $TOKEN"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un 200 con datos de otro usuario/recurso (o una acción aceptada sobre un recurso ajeno) confirma el IDOR. Un 403/404 descarta ese endpoint concreto.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica todos los endpoints que reciben un ID de recurso (usuario, pedido, factura, mensaje).
- Crea o consigue dos identidades si el reto lo permite; compara la misma llamada con cada una.
- Cambia el ID de un recurso propio por uno ajeno y observa la respuesta completa, no solo el código HTTP.
- Prueba también acciones de escritura (PUT/DELETE) sobre IDs ajenos, no solo lectura (GET).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El backend revalida en cada petición que el recurso pertenece al usuario autenticado (403/404 al cambiar el ID).
- El ID no identifica un recurso privado (es público por diseño, como el ID de un producto en un catálogo).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Cambias el ID de un recurso en la URL o el JSON y accedes a datos o acciones de otro usuario porque el backend no revalida el permiso. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (idor-bola)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Seguridad de JWT

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

JWT transporta claims firmados; la seguridad depende de validar firma, algoritmo, audiencia y tiempos.

Aceptar tokens manipulados o mal emitidos permite cambiar identidad, permisos o caducidad.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la API usa tokens con tres segmentos separados por puntos.

Cómo identificarla

- alg
- exp
- sub
- role
- aud

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Decodifica header y payload sin alterar el token original y registra los claims observados.

Evidencia esperada: Conoces el algoritmo, claims sensibles y comprobaciones que debes intentar validar en laboratorio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
python3 -c "import base64,json; t='$TOKEN'.split('.')[1]; print(json.loads(base64.urlsafe_b64decode(t+'*(-len(t)%4)))")"
```

Objetivo y contexto: Inspecciona claims localmente.

Resultado esperado: Payload JSON del token.

Capa 2: prueba documentada

```
jwt_tool $TOKEN
```

Objetivo y contexto: Enumera comprobaciones habituales sobre el JWT.

Resultado esperado: Hallazgos y pruebas sugeridas.

Anatomía de los payloads

La primera prueba es `python3 -c "import base64,json; t='$TOKEN'.split('.')[1]; print(json.loads(base64.urlsafe_b64decode(t+'*(-len(t)%4)))")`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Conoces el algoritmo, claims sensibles y comprobaciones que debes intentar validar en laboratorio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Decodifica header y payload.
- Comprueba expiracion y audiencia.
- Valida que cambios invaliden la firma.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No aplica si la sesion usa exclusivamente cookies opacas sin JWT.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Inspeccionar claims, algoritmo, expiracion y validacion de tokens JWT. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(jwt-security\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Inclusión local de ficheros (LFI)

Objetivos de aprendizaje

Distinguir selección de ruta, lectura e inclusión; reconstruir base y sufijo; identificar el componente que interpreta el fichero; adaptar profundidad, encoding o wrapper solo con evidencia.

Prerrequisitos

Rutas relativas y canónicas, percent-encoding, permisos Linux, HTTP y método de hipótesis.

Fundamentos técnicos

Path traversal es la capacidad de salir de una ubicación prevista mediante una ruta controlada. Lectura arbitraria describe el efecto de un sink como `readfile()` u `open()`. LFI implica que una aplicación incluye un fichero local mediante un mecanismo como `include`; en PHP, `include` incluye y evalúa el fichero, pero el código ejecutable debe aparecer en etiquetas PHP válidas. Por tanto:

control de ruta != lectura garantizada != `include` != ejecución controlada

Cada flecha requiere precondiciones y evidencia propias.

Modelo mental

entrada -> decode -> prefijo + valor + sufijo -> normalización -> permisos
-> open/read/include -> bytes servidos o interpretación -> respuesta

Superficie de ataque

Parámetros de página, idioma, tema, plantilla, documento o descarga. También rutas almacenadas en base de datos y cabeceras que seleccionan localización. El nombre `file` no revela el sink.

Cómo identificarla

- Un error muestra la ruta construida, prefijo o sufijo.
- Un fichero marcador conocido produce contenido inequívoco y un control inexistente produce un error distinto.
- En un `include`, texto fuera de etiquetas puede mostrarse mientras código válido se evalúa.
- Un wrapper solo es relevante si el runtime, la función y la configuración lo soportan.

Preguntas que debo hacerme

1. ¿Cuál es la base y cuántos niveles necesito resolver?
2. ¿La aplicación añade extensión o prefijo?
3. ¿El sink lee bytes o incluye/evalúa?
4. ¿Qué identidad y permisos tiene el proceso?
5. ¿Cuántas decodificaciones y normalizaciones ocurren?

Prueba mínima

Usa un fichero marcador del laboratorio con contenido conocido y un hermano inexistente. No empieces por credenciales ni por ejecución. Un error con ruta absoluta es una pista sobre construcción; no confirma lectura.

Construcción progresiva del payload

Escenario:

```
include __DIR__ . '/views/' . $_GET['page'] . '.php';
```

Entrada normal home produce `/srv/app/views/home.php`. Para alcanzar `/srv/app/markers/probe.php`, la ruta debe subir un nivel desde `views`, entrar en `markers` y tener en cuenta que `.php` ya se añade. La construcción es

../markers/probe, no una cantidad arbitraria de ../.

Si el error revela otra base, se recalcula. Si el proceso no puede leer el fichero, aumentar profundidad no adapta el problema.

Anatomía de los payloads

- Contexto de entrada: query page.
- Sintaxis original: `include '/srv/app/views/' + page + '.php'`.
- Entrada controlada: segmento central.
- Transformaciones conocidas: URL decode una vez.
- Parser final: resolución de ruta y `include` de PHP.
- Sink: inclusión/evaluación local.
- Primitiva: incluir un marcador PHP conocido fuera de views.
- Payload mínimo: `../markers/probe`.
- Significado de cada componente: `..` selecciona `/srv/app`; `markers/probe` selecciona objetivo; el sufijo completa `.php`.
- Resultado esperado: marcador emitido por el fichero.
- Control negativo: `../markers/missing`.
- Restricción observada: el sufijo `.php` se añade.
- Por qué falla la variante básica: `../../etc/passwd` se convierte en `/srv/etc/passwd.php`, objetivo y sufijo incorrectos.
- Hipótesis de adaptación: usar un objetivo compatible con el sufijo o demostrar un mecanismo que transforme la lectura.
- Payload adaptado: `../markers/probe`.
- Por qué debería funcionar: la ruta normalizada existe y es legible.
- Evidencia: salida única del marcador y error diferente para el control.

Cuándo no funcionaría: allowlist de vistas, canonicalización dentro de base, permisos insuficientes o sink de lectura distinto.

Variaciones según el contexto

Sink	Resultado mínimo	¿Interpreta?
<code>readfile(path)</code>	bytes del fichero	no por sí mismo
<code>open(path)</code> + respuesta	bytes si la app los devuelve	no
<code>include(path)</code> PHP	salida y evaluación de PHP válido	sí, según contenido
servidor estático	recurso bajo raíz autorizada	depende del handler, no del nombre

`php://filter/convert.base64-encode/resource=...` puede transformar un recurso en PHP cuando el wrapper y el filtro están disponibles. No elimina automáticamente un sufijo añadido ni existe en otros runtimes.

Filtros y bypasses

Clasifica el control: rechazo textual de ../, normalización canónica, allowlist de identificadores, comprobación de prefijo final o permisos. Solo una validación antes de una decodificación posterior justifica probar una representación codificada equivalente.

Evidencias de confirmación

Contenido inequívoco del fichero o salida de un marcador incluido, con control inexistente. Un warning confirma que se intentó resolver una ruta; no que el objetivo se leyó ni que su contenido se interpretó.

Escalado de impacto

Determinar base, sufijo y sink; confirmar un marcador; mapear permisos; leer solo ficheros necesarios del laboratorio; estudiar interpretación únicamente si se controla contenido de un fichero que el sink incluye.

Errores frecuentes

- Llamar LFI a toda descarga con nombre controlado.
- Suponer que muchos `../` siempre ayudan.
- Ignorar el sufijo añadido.
- Usar wrappers sin comprobar runtime y configuración.
- Confundir warning, lectura y ejecución.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Experimento discriminatorio
error termina en <code>.php</code>	sufijo añadido	objetivo existente con y sin extensión en entrada
ruta absoluta ignora el prefijo	función acepta ruta absoluta	comparar marcador absoluto y relativo autorizado
<code>../</code> se elimina una vez	filtro textual no recursivo	observar valor posfiltro; no probar variantes a ciegas
objetivo existe pero <code>Permission denied</code>	identidad sin lectura	comprobar UID y permisos, no aumentar profundidad
fuelle PHP no aparece	<code>include</code> lo evalúa	marcador de texto y fichero con salida conocida
wrapper se imprime o falla como esquema	sink/runtime no lo reconoce	confirmar PHP, función y wrappers habilitados

Mitigaciones

Mapa fijo de identificadores a plantillas, rutas generadas por la aplicación, canonicalización y comprobación de pertenencia a la base, privilegio mínimo y errores no verbosos. Validar extensión sin comprobar la ruta final no resuelve traversal.

Relación con pentesting y certificaciones

Se espera justificar la ruta normalizada y el sink. El éxito no se mide por alcanzar `/etc/passwd`, sino por explicar qué primitiva se confirmó.

Caso guiado

Caso A — Básico: una vista fuera del directorio

`page=home` carga `/opt/site/views/home.php`. `page=missing` muestra `include(/opt/site/views/missing.php)`. Existe un marcador de laboratorio en `/opt/site/markers/probe.php`.

Observación: el error revela base y sufijo. Qué sé: se construye una ruta de `include`. Hipótesis: el valor admite segmentos relativos o se aplica `basename`. Experimento: `../markers/probe` y control `../markers/missing`.

Predicción: si se resuelve relativo, el primero emite el marcador; con basename, ambos quedan en views. Resultado: aparece INCLUDE_OK; el control genera warning. Conclusión: inclusión local fuera de la base. Siguiendo paso: registrar permisos y detenerse antes de buscar ejecución adicional.

Caso de adaptación

Caso B — El objetivo correcto recibe un sufijo

`../../../../etc/passwd` produce warning sobre `/opt/etc/passwd.php`.

Observación: profundidad y sufijo no coinciden con la hipótesis. Qué sé: `.php` se añade y la base está en `/opt/site/views`. Hipótesis: no puede leerse ese objetivo con esta construcción; quizá existe un wrapper compatible o un fichero `.php` útil del laboratorio. Experimento: primero un marcador `.php` conocido; después, solo si PHP y wrappers están confirmados, comprobar si el wrapper llega intacto al sink. Resultado: el marcador funciona; el wrapper es rechazado por allowlist de esquema. Conclusión: se confirma LFI de `.php`, no lectura arbitraria de cualquier fichero. Siguiendo paso: delimitar impacto real.

Caso C — Transferencia: selector de idioma

Una cookie `lang=es` carga textos. Al enviar `lang=../../../../markers/probe`, aparece INCLUDE_OK; el título del ejercicio no revela la técnica.

Resolución: cookie -> concatenación `/translations/` -> include -> fichero. El mismo control de ruta aparece fuera de `page=`. Un control inexistente separa inclusión de contenido cacheado.

Caso D — Falso positivo: error de plantilla simulado

`page=../../../../x` devuelve una página que imprime literalmente `failed to open stream: /views/../../../../x.php`, pero el mismo texto aparece en el JavaScript del frontend y no cambia con rutas distintas.

Observación: mensaje compatible con PHP. Hipótesis: warning backend o simulación client-side. Experimento: repetir sin JavaScript, cambiar marcador único y revisar respuesta HTTP cruda. Resultado: el servidor siempre devuelve el mismo JSON y el navegador compone el texto. Conclusión: no hay evidencia de filesystem ni include. Siguiendo paso: cerrar la hipótesis.

Ejercicios

1. Normaliza `/srv/app/views/../../../../markers/a.php` y explica cada segmento.
2. Diseña un control que distinga `readfile` de `include` usando dos marcadores seguros.
3. Un wrapper falla con "scheme not allowed": enumera tres causas y una prueba para cada una.
4. Explica por qué `Permission denied` confirma una ruta intentada, no lectura.

Resumen

LFI exige identificar construcción de ruta y sink. Traversal, lectura, include e interpretación son capacidades distintas; cada una se confirma por separado.

Chuleta operativa

1. Base y working directory.
2. Prefijo y sufijo.
3. Decodificaciones.
4. Ruta normalizada.
5. Identidad y permisos.
6. Read frente a include.
7. Marcador y control inexistente.
8. Wrapper solo con evidencia.

Referencias

- [PortSwigger: path traversal](#)
- [PHP: include](#)
- [PHP: wrappers](#)
- [Concepto interno](#) (1-f1)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).

De LFI a ejecucion (RCE)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Es el salto de 'puedo leer' a 'puedo ejecutar'. Una lfi por si sola solo lee, pero si consigues que el fichero incluido contenga código PHP que tu has metido antes, ese código se ejecuta con los permisos del servidor web y obtienes ejecución remota de comandos.

include no solo lee: interpreta PHP. Si logras escribir `<?php system($_GET['c']); ?>` en algún sitio del servidor y luego apuntas la LFI a ese sitio, el include ejecuta tu payload. Los sitios clásicos donde puedes 'inyectar' ese contenido son ficheros que el servidor escribe con datos tuyos: logs de acceso, ficheros de sesión, cabeceras guardadas o subidas de fichero mal filtradas.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ya confirmaste lectura con LFI y además existe algún fichero cuyo contenido puedas influir: el User-Agent acaba en un access.log legible, hay ficheros de sesión en /var/lib/php, o hay un formulario de subida. Sin un punto de inyección controlable, la LFI se queda en lectura.

Cómo identificarla

- Puedes leer un log (access.log, auth.log) vía la LFI: el log refleja datos que tu envías, como el User-Agent.
- Existe una subida de ficheros que no valida bien la extensión o el contenido.
- Puedes fijar el valor de una cookie de sesión y localizar su fichero en disco.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Manda un User-Agent con PHP mínimo (`<?php system($_GET['c']); ?>`) y luego incluye el access.log con `?c=id`.

Evidencia esperada: La salida de id aparece incrustada entre las líneas del log. Eso confirma ejecución, no solo lectura.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -s '$URL/?page=../../../../../../var/log/apache2/access.log&c=id'
```

Objetivo y contexto: Log poisoning: antes mandas PHP en tu User-Agent (queda escrito en el access.log) y ahora incluyes ese log vía la LFI. El servidor interpreta tu PHP y ejecuta el comando de ?c=.

Resultado esperado: La salida de id incrustada entre las líneas del log confirma ejecución. Fijate en el usuario (www-data): es con quien tendrás la shell al escalar a reverse shell.

Anatomía de los payloads

La primera prueba es `curl -s '$URL/?page=../../../../../../var/log/apache2/access.log&c=id'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La salida de id aparece incrustada entre las líneas del log. Eso confirma ejecución, no solo lectura.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Elige el vector de inyección: log poisoning (mandas PHP en el User-Agent y lo ejecutas incluyendo el log), sesión, o subida.
- Inyecta un payload mínimo que ejecute un comando parametrizado, y confirma con id o whoami que se ejecuta.
- Convierte la ejecución de un comando en una shell interactiva: lanza una reverse shell (reverse-vs-bind) y estabilízala (tty-stabilization).
- Con shell ya dentro, cambia de fase: orientate y busca escalada (privesc-modelo). Los comandos concretos están en la sección de práctica (ver comandos).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay ningún fichero en el servidor cuyo contenido controles (logs no legibles, sin uploads, sin sesiones en disco).

- El servidor no interpreta PHP en el fichero incluido (p.ej. solo lo muestra como texto).
- open_basedir u otra restriccion impide llegar a la ruta del fichero envenenado.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Leer ficheros no es el final: si logras que el servidor incluya contenido que tu controlas, ejecutas código. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- Concepto fuente de THM Fieldbook (lfi-a-rce)
- Referencia técnica externa

Navegación

Anterior: Construcción de payloads. Índice: curso. Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Análisis de MFA y OTP

Objetivos de aprendizaje

Modelar el estado pre-MFA y autenticado, comprobar binding de usuario/sesión/desafío, detectar replay y manipulación de estado, y descartar bypasses aparentes.

Prerrequisitos

Sesiones, autenticación, autorización, HTTP multipart/JSON y controles de repetición.

Fundamentos técnicos

El servidor debe verificar el factor y realizar la transición de estado. Un OTP debe tener vigencia limitada, límites de intentos y uso único. El estado no debe depender de una bandera enviada por el cliente. Además, desafío, usuario y sesión deben estar ligados para impedir que un código válido se aplique a otro flujo.

Modelo mental

```
credencial primaria -> sesión pre-MFA + desafío(user, session, TTL)
                    -> verificación OTP -> consumir desafío -> rotar/elevar sesión
                    -> autorización del recurso
```

Superficie de ataque

Login, recuperación, cambio de factor, "remember device", reautenticación de acciones sensibles, resend y endpoints API equivalentes al paso visual.

Cómo identificarla

- Acceso directo al recurso con sesión pre-MFA.
- Bandera cliente cambia la transición sin OTP válido.
- Código de A funciona en sesión B.
- OTP aceptado más de una vez o después de resend/expiración.
- Intentos ilimitados o contadores no ligados a cuenta/desafío.

Preguntas que debo hacerme

1. ¿Qué estado tiene la sesión antes y después?
2. ¿A qué usuario, sesión y desafío está ligado el OTP?
3. ¿Se consume al usarlo y al reenviar?
4. ¿Qué recurso prueba realmente la transición?
5. ¿Hay fallback o recuperación más débil?

Prueba mínima

Con cuentas de laboratorio, comparar OTP inválido, válido, reutilizado, expirado y de otra sesión. Verificar el recurso protegido, no solo la redirección.

Construcción progresiva del payload

La secuencia mínima conserva cookie pre-MFA, envía código inválido con y sin campos adicionales y consulta el recurso protegido. Después usa un código válido una vez y repite exactamente la solicitud para probar replay.

Anatomía de los payloads

- Contexto de entrada: JSON /mfa/verify con cookie pre-MFA.
- Sintaxis original: {"otp":"000000","verified":false}.

- Entrada controlada: OTP y bandera cliente.
- Transformaciones conocidas: JSON decode y lookup de desafío.
- Parser final: máquina de estados de autenticación.
- Sink: elevación de sesión.
- Primitiva: elevar con OTP inválido cambiando solo `verified`.
- Payload mínimo: `{"otp": "000000", "verified": true}`.
- Significado de cada componente: código deliberadamente inválido; bandera reclama un estado.
- Resultado esperado: rechazo y sesión pre-MFA sin cambios en una implementación segura.
- Control negativo: mismo OTP con `false` y acceso directo al recurso.
- Restricción observada: endpoint ignora el código cuando `verified=true`.
- Por qué falla la variante básica: enviar solo OTP inválido no alcanza la rama defectuosa.
- Hipótesis de adaptación: mass assignment/estado cliente confiado.
- Payload adaptado: añadir solo la propiedad observada en el esquema.
- Por qué debería funcionar: el backend vulnerable la enlaza al objeto de estado.
- Evidencia: nueva sesión accede al recurso pese a OTP inválido.
- Cuándo no funcionaría: DTO cerrado y estado derivado exclusivamente del verificador.

Variaciones según el contexto

TOTP, código por correo, push y WebAuthn tienen propiedades distintas. El análisis común es binding, frescura, consumo, intentos y transición. “Recordar dispositivo” crea otro autenticador persistente que debe evaluarse por separado.

Filtros y bypasses

Cambiar nombres de bandera al azar no es metodología. Deriva campos del tráfico o esquema, modifica uno y comprueba estado server-side. Rate limiting, replay y bypass de transición son hallazgos distintos.

Evidencias de confirmación

Acceso al recurso o sesión elevada sin factor válido; aceptación cruzada entre usuarios/sesiones; reutilización del mismo OTP cuando debería ser de un solo uso. Ver la pantalla de OTP o saltarla visualmente no basta.

Escalado de impacto

Probar solo cuentas y desafíos propios, medir límites sin agotamiento abusivo y evaluar recuperación/cambio de factor con datos de laboratorio.

Errores frecuentes

- Concluir bypass por recibir 302.
- No conservar la cookie pre-MFA correcta.
- Confundir ausencia de rate limit con bypass inmediato.
- Reutilizar un OTP dentro de una ventana sin conocer política de consumo.
- Probar código de otro usuario sin controlar el binding de sesión.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
verified=true ignorado	DTO cerrado o nombre incorrecto	esquema/traza y propiedad desconocida como control
dashboard carga, API privada da 401	solo salto visual	consultar recurso protegido crudo
OTP válido de A falla en B	binding correcto o TTL	misma sesión A como control
OTP se acepta dos veces	replay o dos desafíos	comparar challenge ID y evento de consumo
resend deja ambos códigos válidos	invalidación incompleta	usar viejo/nuevo en orden controlado
intentos se reinician con cookie	contador ligado a sesión	misma cuenta con sesión nueva, sin exceder límites

Mitigaciones

Estado server-side, DTO cerrado, binding a usuario/sesión/desafío, TTL corto, uso único, límites por cuenta y desafío, rotación de sesión tras MFA, recuperación equivalente y registros sin OTP.

Relación con pentesting y certificaciones

La competencia es demostrar una transición inválida y su efecto de autorización, no adivinar códigos.

Caso guiado

Caso A — Básico: estado cliente confiado

Con OTP inválido, verified=false rechaza y verified=true entrega una cookie nueva que accede a /account.

Observación: una propiedad cambia estado. Qué sé: el código sigue inválido. Hipótesis: backend confía en bandera o existe caché. Experimento: cookies separadas, token de desafío único y acceso directo. Resultado: solo la bandera eleva la sesión. Conclusión: bypass de MFA por manipulación de estado. Siguiendo paso: documentar binding y rotación.

Caso de adaptación

Caso B — La bandera no funciona, pero el código se reutiliza

El DTO rechaza propiedades extra. Un OTP válido se acepta dos veces en dos sesiones del mismo usuario.

Observación: mass assignment descartado; posible replay. Hipótesis: mismo desafío compartido o consumo ausente. Experimento: registrar challenge ID, usar código una vez y repetir antes de expirar. Resultado: mismo challenge aceptado dos veces. Conclusión: fallo de uso único, no manipulación de parámetro. Siguiendo paso: probar invalidación tras resend.

Caso C — Transferencia: cambio de teléfono

El cambio de factor crea un desafío, pero una sesión pre-MFA de login puede enviarlo y modificar el número. El título no revela la técnica.

Resolución: sesión pre-MFA -> endpoint de gestión -> autorización insuficiente. La primitiva transferida es una transición sensible accesible en estado incorrecto, no un OTP adivinado.

Caso D — Falso positivo: ruta /dashboard

Tras OTP inválido, el servidor redirige a /dashboard, cuyo HTML carga, pero todas las API privadas responden 401 y la página muestra "sesión incompleta".

Experimento: recurso protegido y estado /me. Resultado: sesión sigue pre-MFA. Conclusión: navegación no equivale a bypass.

Ejercicios

1. Diseña una matriz OTP inválido/válido/replay/expirado y sesión A/B.
2. Distingue rate limiting débil de bypass de transición.
3. Explica cómo probar binding sin usar cuentas ajenas.
4. Modela resend y consumo del desafío.

Resumen

MFA es una máquina de estados ligada a un desafío. La evidencia es el estado autorizado obtenido, no la pantalla ni la redirección.

Chuleta operativa

1. Sesión pre-MFA.
2. Challenge ID, usuario y TTL.
3. OTP inválido/válido/replay.
4. Propiedades extra derivadas del esquema.
5. Recurso protegido.
6. Rotación, consumo y límites.

Referencias

- [OWASP MFA Cheat Sheet](#)
- [NIST SP 800-63B](#)
- [Concepto interno](#) (mfa-otp-bypass)

Navegación

Anterior: [File upload](#). Siguiente: [Autenticación y sesiones](#). Índice: [curso](#).

OAuth, OIDC y CORS

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

OAuth delega autorización y OIDC añade identidad sobre OAuth. CORS decide que origen puede leer respuestas desde el navegador; no sustituye los controles del servidor.

Un `redirect_uri` flexible, un `state` ausente o un CORS que refleja cualquier Origen puede filtrar tokens o permitir acciones desde un origen atacante.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ves `/authorize`, `/token`, `.well-known/openid-configuration`, `access_token` en navegador o cabeceras `Access-Control-Allow-Origin`.

Cómo identificarla

- `redirect_uri`
- `state`
- `code_challenge`
- `issuer`
- `Access-Control-Allow-Origin`
- `credentials: true`

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara `redirect_uri`, `state`, PKCE, `issuer/audience` y respuesta CORS con orígenes permitidos y no permitidos.

Evidencia esperada: Tienes una matriz de origen, token y recurso que demuestra o descarta fuga y autorización rota.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS $URL/.well-known/openid-configuration
```

Objetivo y contexto: Obtiene issuer, endpoints y capacidades declaradas por OIDC.

Resultado esperado: JSON de discovery con `authorization_endpoint` y `token_endpoint`.

Capa 2: prueba documentada

```
curl -i -H 'Origin: https://attacker.example' $URL/api/me
```

Objetivo y contexto: Comprueba si un origen no confiable recibe cabeceras CORS permisivas.

Resultado esperado: `Access-Control-Allow-Origin` y `Allow-Credentials` para comparar.

Anatomía de los payloads

La primera prueba es `curl -sS $URL/.well-known/openid-configuration`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tienes una matriz de origen, token y recurso que demuestra o descarta fuga y autorización rota.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Descubre metadata OIDC.
- Valida state, PKCE y redirect exacto.
- Prueba CORS desde origen controlado.
- Comprueba permisos del token en cada recurso.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay flujo OAuth/OIDC ni endpoints de autorización.
- La prueba se limita a una API server-to-server sin navegador y sin CORS.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Auditar redirecciones, tokens y políticas CORS sin confundir autenticación con autorización. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (oauth-oidc-cors)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Inyección SQL (SQLi)

Objetivos de aprendizaje

Reconstruir la consulta probable, identificar el contexto de inserción, construir un oráculo mínimo, distinguir señales de falsos positivos y adaptar sintaxis solo después de obtener evidencia sobre el motor.

Prerrequisitos

Flujo source-transformación-sink, sintaxis SQL básica, parámetros preparados, HTTP y método de hipótesis.

Fundamentos técnicos

Existe SQLi cuando datos controlados alteran la estructura de una sentencia SQL. La causa no es “aceptar comillas”, sino mezclar estructura y valores en el mismo texto antes de que lo analice el motor. Un valor parametrizado sigue siendo dato aunque contenga SQL; un identificador dinámico, como una columna de orden, requiere seleccionar entre expresiones permitidas por la aplicación.

Modelo mental

entrada -> decode/coerción -> concatenación o binding -> parser SQL
-> plan/ejecución -> filas, error, booleano, tiempo u OOB

Antes de escribir un payload responde: ¿el valor está dentro de un literal, es una expresión numérica, ocupa una lista, un ORDER BY o ni siquiera llega a SQL?

Superficie de ataque

Buscadores, filtros, login, IDs, cookies de preferencias, cabeceras usadas en auditoría y cuerpos JSON pueden alimentar consultas. El nombre del parámetro no importa; importa el flujo hasta el motor.

Cómo identificarla

- Parejas verdadera/falsa producen diferencias reproducibles dependientes de la condición.
- Un error atribuido al motor revela contexto, pero por sí solo es un indicio.
- Una demora confirma un canal solo si escala con el valor solicitado, se repite y se separa de controles.
- UNION solo es útil si la consulta devuelve filas, coincide el número de columnas y hay tipos compatibles.

Preguntas que debo hacerme

1. ¿Qué consulta explica la respuesta normal?
2. ¿El punto de inserción es textual, numérico o estructural?
3. ¿Qué sufijo queda después de mi entrada?
4. ¿Qué canal observable tengo?
5. ¿Qué comportamiento alternativo produciría la misma señal?

Prueba mínima

Una comilla es una sonda de sintaxis, no una confirmación. La prueba mínima preferida es una pareja válida para el contexto que cambie solo una condición, acompañada de un valor inexistente y repetición.

Construcción progresiva del payload

Escenario base:

```
SELECT id, name, price FROM products WHERE category = '<entrada>' AND visible = 1
```

1. Entrada válida: gifts.
2. Reconstrucción: el valor está dentro de comillas y queda el sufijo ' AND visible = 1.

3. Primitiva: variar una condición sin extraer datos.
4. Variante verdadera: cerrar el literal, añadir una condición verdadera y neutralizar el sufijo según el motor confirmado.
5. Variante falsa: misma estructura, una condición falsa.
6. Solo si existe un canal de filas se estudia UNION; primero número de columnas, luego compatibilidad de tipos y columna reflejada.

No se encadenan detección, identificación de motor y extracción en una única cadena: cada paso debe resolver una incertidumbre.

Anatomía de los payloads

- Contexto de entrada: query category.
- Sintaxis original: `WHERE category = '<entrada>' AND visible = 1.`
- Entrada controlada: interior de un literal textual.
- Transformaciones conocidas: URL decode una vez, sin coerción.
- Parser final: PostgreSQL confirmado en el laboratorio por error de versión controlado.
- Sink: ejecución de `SELECT`.
- Primitiva: oráculo booleano.
- Payload mínimo: `gifts' AND '1'='1' --` , emparejado con `gifts' AND '1'='2' --` .
- Significado de cada componente: `gifts'` cierra el valor; `AND` añade condición; la comparación cambia verdad/falsedad; `--` neutraliza el sufijo en este contexto.
- Resultado esperado: la pareja produce presencia/ausencia estable del producto.
- Control negativo: categoría inexistente y petición original.
- Restricción observada: el sufijo provoca error si no se neutraliza.
- Por qué falla la variante básica: `' OR '1'='1` deja comillas y `AND visible=1` en una estructura distinta de la asumida.
- Hipótesis de adaptación: reconstruir y neutralizar exactamente el sufijo.
- Payload adaptado: la pareja anterior.
- Por qué debería funcionar: ambas consultas resultantes son válidas y difieren solo en una condición.
- Evidencia: diferencia reproducible en cuerpo y número de filas.

Cuándo no funcionaría: binding parametrizado, contexto numérico, comentarios distintos o filtro semántico.

Variaciones según el contexto

Contexto probable	Reconstrucción	Primera pareja útil
Numérico	<code>WHERE id = <entrada></code>	<code>12 AND 1=1 / 12 AND 1=2</code>
Texto	<code>WHERE name = '<entrada>'</code>	cerrar y construir dos comparaciones válidas
ORDER BY	<code>ORDER BY <entrada></code>	valores permitidos que revelen orden; no tratar como literal
LIKE	<code>LIKE '%<entrada>'</code>	considerar comodines y ambos delimitadores

Las funciones de versión, concatenación, demora, catálogos y comentarios cambian entre motores. Se consultan después de identificar el motor; no se usan como detector universal.

Filtros y bypasses

Un rechazo puede venir de conversión de tipo, WAF, parser HTTP o motor. Compara forma enviada, valor recibido y consulta final. Sustituir espacios, mayúsculas o encoding solo está justificado si una prueba localiza la comparación que se intenta eludir.

Evidencias de confirmación

La mejor evidencia es causal: dos consultas que solo difieren en una condición producen resultados opuestos de forma repetible. Un 500, una página vacía o una demora aislada no confirman SQLi.

Escalado de impacto

Tras confirmar la primitiva: identificar motor y versión solo si aporta sintaxis; determinar canal; verificar columnas y tipos si procede; demostrar acceso mínimo a un dato no sensible del laboratorio; documentar privilegios de la cuenta. Automatizar no sustituye la confirmación manual ni autoriza un volcado indiscriminado.

Errores frecuentes

- Usar payload textual en contexto numérico.
- Asumir que `information_schema`, comentarios o funciones son iguales en todos los motores.
- Interpretar 500 como confirmación.
- Usar UNION sin conocer columnas, tipos o canal de renderizado.
- Cambiar condición, comentario y encoding simultáneamente.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Experimento discriminatorio
' produce 500, pero cualquier símbolo también	conversión o error genérico	comparar caracteres inocuos y revisar fase del error
1 AND 1=1 y 1 AND 1=2 son iguales	binding, coerción o canal sin filas	valor inexistente, log de consulta y otro canal controlado
ORDER BY 4 falla	índice fuera de rango o lógica de aplicación	comparar 1..4 y conservar cuerpos completos
UNION siempre falla	columnas/tipos/contexto/motor	reducir a NULL, reconstruir sufijo y consultar sintaxis confirmada
demora variable no proporcional	carga, lock o reintento	alternar controles y dos duraciones en varias muestras
error nombra otro parser	la entrada no llegó a SQL	corregir JSON, tipo o transporte antes de adaptar SQL

Mitigaciones

Consultas preparadas con binding para valores, mapas cerrados para identificadores dinámicos, privilegio mínimo, errores externos no verbosos y pruebas sobre la frontera exacta. Escapar cadenas es frágil y dependiente del motor.

Relación con pentesting y certificaciones

Se evalúa la capacidad de justificar la consulta reconstruida, el oráculo y los controles, no la memoria de cadenas ni la cantidad de datos extraídos.

Caso guiado

Caso A — Básico: filtro de precio numérico

GET /search?max=50 devuelve 12 productos; max=50 AND 1=1 devuelve 12 y max=50 AND 1=2 devuelve 0.

Observación: una pareja lógica cambia filas. Qué sé realmente: el valor acepta sintaxis compatible con una expresión booleana. Hipótesis: SQL concatenado o evaluador previo. Experimento discriminatorio: valor inexistente, expresión propia del SQL estándar y log de consulta del laboratorio. Resultado esperado: SQL concatenado conserva la expresión en WHERE price <= ...; un evaluador enviaría un número calculado. Resultado obtenido: el log muestra price <= 50 AND 1=2. Conclusión: SQLi numérica con canal booleano. Siguiendo paso: identificar motor antes de usar funciones específicas.

Caso de adaptación

Caso B — La comilla no funciona

En /item?id=7, ' responde 400 integer required, pero 7-1 devuelve el artículo 6.

Observación: la comilla muere antes del sink; aritmética cambia el objeto. Qué sé: existe coerción parcial o contexto numérico. Hipótesis: H1, validación permite expresiones y SQL las evalúa; H2, la aplicación evalúa y parametriza. Experimento: pareja 7 AND 1=1 / 7 AND 1=2 y trazas del laboratorio. Predicción: H1 produce diferencia en filas; H2 rechazará AND o enviará un entero. Resultado: diferencia estable y SQL final visible. Conclusión: la adaptación correcta elimina comillas; no busca codificarlas. Siguiendo paso: documentar el oráculo.

Caso C — Transferencia: preferencia de orden

Una cookie sort=price cambia el orden del catálogo. No se nombra la vulnerabilidad. El alumno debe mapear cookie -> selección de columna -> ORDER BY. Un intento con comillas no cambia nada. Cambiar entre price y name sí altera el orden; un valor fuera de catálogo produce un error de columna del motor.

Resolución: la entrada no ocupa un literal sino una posición estructural. El experimento correcto compara columnas existentes y un valor inválido; la defensa es un mapa cerrado de nombres públicos a expresiones SQL, no intentar parametrizar el identificador como valor.

Caso D — Falso positivo: el 500 de la comilla

Una API declara id como UUID. Cualquier valor con longitud distinta de 36, incluida ', produce el mismo 500 por una excepción del validador.

Observación: la comilla causa error. Qué sé: solo que el input es inválido. Hipótesis: SQLi o validación defectuosa. Experimento: varios UUID inválidos de igual longitud, un UUID válido inexistente y logs de capa. Resultado: todos los inválidos fallan antes del repositorio; el válido inexistente produce 404. Conclusión: no hay evidencia de SQLi. Siguiendo paso: reportar manejo de errores solo si entra en alcance.

Ejercicios

1. Reconstruye SELECT ... WHERE code='<entrada>' AND tenant=? y diseña una pareja que no altere el tenant.
2. Una demora de 3 s aparece en 2 de 10 payloads y 3 de 10 controles: decide qué medir ahora.
3. Explica por qué UNION SELECT NULL, NULL no prueba nada si el resultado no se renderiza.
4. Diseña una prueba que distinga coerción a entero de SQL numérico concatenado.

Resumen

SQLi se demuestra reconstruyendo contexto y obteniendo un oráculo causal. Los payloads se derivan de la consulta y del motor confirmado; no se eligen de una lista.

Chuleta operativa

1. Entrada válida y consulta probable.
2. Contexto textual, numérico o estructural.

3. Sufijo restante.
4. Pareja verdadera/falsa y control inexistente.
5. Canal reproducible.
6. Motor antes de sintaxis específica.
7. Impacto mínimo y mitigación causal.

Referencias

- [PortSwigger: SQL injection](#)
- [PortSwigger: sintaxis específica por motor](#)
- [OWASP: SQL Injection Prevention](#)
- [Concepto interno \(sqli\)](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).

Server-Side Request Forgery (SSRF)

Objetivos de aprendizaje

Demostrar quién realiza una petición, separar validación de resolución y conexión, diagnosticar filtros de destino y reconocer cuándo la salida revela el cliente HTTP subyacente.

Prerrequisitos

URL, DNS, loopback, redirects, parsers, callbacks controlados y método de hipótesis.

Fundamentos técnicos

SSRF permite que una aplicación server-side solicite una ubicación no prevista. Una URL reflejada, un redirect o una carga del navegador no bastan. La primitiva mínima es una conexión atribuible al servidor hacia un destino controlado.

Modelo mental

URL original -> parser de validación -> DNS/IP/política -> cliente HTTP
-> redirect -> nueva resolución/política -> conexión -> canal de respuesta

Validar solo la representación original no garantiza el destino efectivo. La política debe considerar esquema, host normalizado, IP resuelta, puerto y redirects.

Superficie de ataque

Importadores, webhooks, avatares remotos, previews, renderizadores PDF, validadores de enlaces, feeds, analítica y formatos que referencian recursos externos.

Cómo identificarla

- Callback único recibido al ejecutar la función, aun sin navegador.
- IP origen y User-Agent compatibles con infraestructura server-side.
- Diferencias reproducibles entre destino que responde, cerrado e inexistente.
- Contenido o errores del destino aparecen en la respuesta.

Preguntas que debo hacerme

1. ¿Quién inicia realmente la conexión?
2. ¿Qué forma valida la aplicación y qué forma consume el cliente?
3. ¿Cuándo resuelve DNS y qué IP conecta?
4. ¿Sigue redirects y revalida cada salto?
5. ¿Qué protocolos soporta realmente el cliente detectado?

Prueba mínima

Usa un listener HTTP propio del laboratorio con token único. Guarda la URL sin abrir vistas que carguen recursos y correlaciona tiempo, IP, cabeceras y endpoint. Un control apunta a un puerto cerrado del mismo host.

Construcción progresiva del payload

1. Confirmar callback externo.
2. Distinguir visible de blind.
3. Comparar loopback permitido, bloqueado y destino inexistente solo dentro del laboratorio.
4. Identificar si el filtro actúa sobre texto, host parseado o IP efectiva.

5. Probar redirect controlado y observar si existe segunda resolución.
6. Solo tras reconocer cliente y configuración, estudiar capacidades adicionales.

Anatomía de los payloads

- Contexto de entrada: formulario POST /import, campo url.
- Sintaxis original: cliente solicita la URL completa.
- Entrada controlada: esquema, autoridad, puerto y path.
- Transformaciones conocidas: form decode, parser URL, resolución DNS.
- Parser final: cliente HTTP server-side.
- Sink: conexión de red.
- Primitiva: callback HTTP a listener propio.
- Payload mínimo: `http://listener.lab/ssrf-7f31`.
- Significado de cada componente: `http` selecciona protocolo; `host` resuelve al listener; `token` correlaciona la prueba.
- Resultado esperado: una petición server-side al token.
- Control negativo: puerto cerrado y token distinto no enviado.
- Restricción observada: destinos loopback escritos como `127.0.0.1` se bloquean.
- Por qué falla la variante básica: la validación rechaza esa representación antes del cliente.
- Hipótesis de adaptación: validación textual distinta de la normalización del cliente.
- Payload adaptado: representación alternativa solo después de comprobar cómo la interpreta ese cliente.
- Por qué debería funcionar: ambas formas podrían resolverse a loopback, pero el filtro compara solo una.
- Evidencia: respuesta local y traza de conexión a la IP efectiva.
- Cuándo no funcionaría: política sobre IP canónica, cliente que no acepta la forma o revalidación correcta.

Variaciones según el contexto

SSRF visible devuelve contenido; blind exige callback o señal temporal robusta. Un redirect solo cambia el destino si el cliente lo sigue. Un protocolo alternativo solo existe si el cliente lo implementa y la política lo permite; conocer su nombre no demuestra soporte.

Filtros y bypasses

Clasifica antes de adaptar: esquema no permitido, host no permitido, IP privada, puerto, DNS, redirect o salida. Prueba una variable cada vez. Las representaciones alternativas de IP dependen del parser; nunca se presentan como universales.

Evidencias de confirmación

Callback atribuible al backend o contenido inequívoco obtenido por él. La presencia de `url=`, un error DNS genérico o una demora aislada son indicios.

Escalado de impacto

Determinar alcance de destinos y puertos con pruebas mínimas, observar identidad o cabeceras añadidas, y documentar si el backend accede a recursos no alcanzables externamente. No explorar terceros ni metadatos cloud fuera de un laboratorio preparado.

Errores frecuentes

- Confundir carga de `<img>` con SSRF.
- Asumir que un redirect siempre se sigue.

- Probar muchas representaciones sin conocer parser.
- Inferir curl solo por una barra parecida.
- Confundir SSRF con inyección de argumentos en un cliente CLI.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
listener recibe callback, localhost no	SSRF confirmada con política interna	comparar IP cerrada y bloqueo explícito
127.0.0.1 bloqueado, forma equivalente funciona	validación textual o normalización diferencial	registrar host parseado e IP conectada
callback solo al abrir navegador	carga client-side	guardar sin render y comparar origen
primer host permitido, redirect interno bloqueado	revalidación tras redirect	redirect externo a externo como control
redirect interno funciona	validación solo del primer salto	capturar ambos requests y DNS
aparece salida de curl	posible CLI o wrapper	opción inocua y firma/versionado, sin asumir splitting
file:// bloqueado	política de esquema o cliente sin soporte	comparar error de parser con esquema inventado

Mitigaciones

Allowlist de destinos necesarios, parser único, resolución y comprobación de IP efectiva, bloqueo de redes no permitidas, revalidación tras cada redirect, límites de protocolo/puerto y egress controlado.

Relación con pentesting y certificaciones

La competencia clave es atribuir la conexión y explicar la discrepancia entre validación y destino, no memorizar escrituras de loopback.

Caso guiado

Caso A — Básico: importador de feed

POST /feeds/import acepta url. Un token único llega al listener durante el POST con User-Agent: FeedWorker/2.1, aunque nadie abre la vista.

Observación: callback correlacionado con el importador. Qué sé: un componente server-side solicita la URL.

Hipótesis: worker directo o cola asíncrona. Experimento: dos tokens con tiempos y un destino cerrado. Predicción: la cola puede retrasar, pero mantiene origen y token; el navegador no participa. Resultado: callbacks desde la red del servidor y error diferente para puerto cerrado. Conclusión: SSRF básica confirmada. Siguiente paso: estudiar política de destinos con marcadores inocuos.

Caso de adaptación

Caso B — Representación, parser y herramienta

http://127.0.0.1/ devuelve Private network access blocked. Una forma abreviada de loopback aceptada por el cliente devuelve HTML local. La respuesta incluye % Total, % Received y % Xferd.

Observación: dos textos acaban potencialmente en el mismo destino y aparece una firma compatible con curl. Qué sé: el filtro no trató ambas formas igual; todavía no sé si la salida procede de curl real ni si existe splitting. Hipótesis: H1, regex textual + CLI curl; H2, biblioteca que reproduce el formato; H3, destinos realmente distintos. Experimento: registrar IP conectada, comparar User-Agent, provocar un error de opción inocuo solo si existe evidencia de

argumentos separados y consultar la versión expuesta por el laboratorio. Predicción: H1 muestra misma IP y errores propios de curl; H2 no acepta opciones; H3 conecta distinto. Resultado: misma IP y error oficial de curl al recibir una opción separada por el wrapper. Conclusión: SSRF con validación textual y un segundo riesgo potencial de argumentos. Siguiente paso: estudiar documentación de esa versión; no saltar a un protocolo o fichero final.

Caso C — Transferencia: documento mensual

Una función genera PDF desde una plantilla que contiene un logo remoto. El título no nombra la técnica. Al usar un token de imagen, el callback ocurre durante POST /reports/render, desde el render.

Resolución: entrada en HTML -> parser del render -> cliente de recursos -> conexión. La primitiva es la misma aunque no exista parámetro url. El siguiente experimento compara recurso externo, relativo y bloqueado, sin apuntar a servicios sensibles.

Caso D — Falso positivo: avatar remoto

Guardar avatar_url no produce tráfico; abrir el perfil en Firefox sí, con IP y User-Agent del alumno.

Observación: hay callback. Hipótesis: backend o navegador. Experimento: guardar sin renderizar, solicitar JSON crudo y comparar origen. Resultado: solo el navegador conecta. Conclusión: petición client-side; SSRF descartada. Siguiente paso: evaluar controles de contenido del navegador solo si entra en alcance.

Ejercicios

1. Diseña un control para separar DNS server-side de HTTP server-side.
2. Una URL permitida redirige dos veces: indica qué registrar en cada salto.
3. Explica qué evidencia necesitas antes de probar una opción de curl.
4. Distingue SSRF, open redirect y carga client-side en tres trazas dadas.

Resumen

SSRF se confirma atribuyendo una conexión al servidor. Los bypasses se explican mediante parser, resolución y política; las capacidades del cliente se estudian después de identificarlo.

Chuleta operativa

1. Token único y listener.
2. Origen, tiempo y User-Agent.
3. URL original y parseada.
4. DNS e IP efectiva.
5. Redirect y segunda validación.
6. Cliente/versionado con evidencia.
7. Control cerrado e inexistente.

Referencias

- [PortSwigger: SSRF](#)
- [RFC 3986](#)
- [curl manual](#)
- [Concepto interno \(ssrf\)](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).

Server-Side Template Injection (SSTI)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Muchas apps meten tu entrada dentro de una plantilla (Jinja2, Twig, Freemarker...) para generar HTML dinámico. Si el motor de plantillas EVALUA tu entrada en vez de solo insertarla como texto, puedes inyectar sintaxis de la propia plantilla. Según el motor, eso escala desde leer variables internas hasta ejecutar comandos del sistema.

El programador hace algo como `render_template_string('Hola ' + nombre)` en vez de pasar nombre como variable separada. El motor no distingue 'tu texto' de 'código de plantilla que debo evaluar', igual que en `sqli` o `command-injection` la app no distingue dato de instrucción.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando tu entrada aparece reflejada en la página y el backend usa un motor de plantillas (Jinja2/Python, Twig/PHP, Freemarker o Velocity/Java, ERB/Ruby). Sospecha en campos de nombre, mensajes, plantillas de email o generación de PDF/HTML.

Cómo identificarla

- El campo se refleja en la respuesta tal cual lo mandas.
- Mensajes de error que mencionan Jinja2, Twig, Freemarker, Velocity o un motor de plantillas concreto.
- `{{7*7}}` (o el payload equivalente del motor) se convierte en 49 en la respuesta.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Manda una operación matemática como payload (`{{7*7}}`, `${7*7}` o `<%= 7*7 %>` según sospeches el motor) en el campo reflejado.

Evidencia esperada: La respuesta muestra 49 en vez del texto literal. Eso confirma que el motor está EVALUANDO tu entrada, no solo mostrándola.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/?name={{7*7}}"
```

Objetivo y contexto: Prueba de confirmacion minima para Jinja2/Twig-style. Si el motor evalua, 49 aparece en la respuesta; si no, veras el texto literal.

Resultado esperado: Un '49' en la respuesta HTML confirma SSTI. Si no cambia nada, prueba otras sintaxis: `{{7*7}}` (Freemarker/Velocity) o `<%= 7*7 %>` (ERB).

Capa 2: prueba documentada

```
curl -sS "$URL/?name={{config.__class__.__init__.__globals__['os'].popen('id').read()}}"
```

Objetivo y contexto: Payload de escalada especifico de Jinja2/Flask: navega desde un objeto de config accesible hasta el modulo os para ejecutar comandos.

Resultado esperado: La salida de id (uid=... gid=...) en la respuesta confirma RCE completo, no solo evaluacion de expresiones.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/?name={{7*7}}"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La respuesta muestra 49 en vez del texto literal. Eso confirma que el motor esta EVALUANDO tu entrada, no solo mostrandola.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma con una operacion matematica minima (`{{7*7}}`) antes de nada mas.
- Identifica el motor exacto: cada uno tiene payloads de deteccion distintos (Jinja2 vs Twig vs Freemarker dan resultados diferentes a la misma sintaxis).

- Una vez identificado, escala con el payload de RCE específico del motor (en Jinja2, via `__class__.__init__.__globals__` hasta llegar a `os.popen`).
- Con ejecución de comandos, confirma con `id` y salta a conseguir shell: mira `reverse-vs-bind`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Tu entrada se refleja pero SIN evaluar (aparece literal `{{7*7}}` en el HTML): es solo XSS/reflexión, no SSTI.
- La app no usa motor de plantillas (renderiza HTML estático o via un framework que sanea siempre).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada se evalúa como código de plantilla en vez de mostrarse como texto; según el motor, eso llega a ejecución de comandos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(ssti\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Atacar WordPress

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

WordPress es el CMS mas comun en rooms. Su nucleo se actualiza solito, asi que rara vez es el vector. El fallo vive en los plugins y temas de terceros (miles, muchos sin mantener) o en credenciales debiles del panel /wp-admin, desde donde se llega a ejecucion editando un tema.

Cada plugin es codigo de terceros con su propia superficie; una version vulnerable de un plugin popular es explotable en masa. Y el panel de admin permite editar los .php del tema: si entras, tienes RCE casi directo. wpscan automatiza detectar version de core, plugins, temas y usuarios.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando whatweb o /wp-login.php delatan WordPress. Encaja tras el fingerprint web, antes de intentar bugs genericos.

Cómo identificarla

- Rutas /wp-content/, /wp-login.php, /wp-json/ o el meta generator con la version.
- wpscan lista un plugin con CVE conocido o una version vieja.
- Usuarios enumerados via /?author=1 o la API REST -> objetivos de spraying.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

wpscan --url \$URL --enumerate vp,u y revisa si algun plugin sale marcado [!] vulnerable.

Evidencia esperada: Version del core, lista de plugins con su version, y usuarios enumerados via /?author= o REST API.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
wpscan --url $URL --enumerate vp,u --plugins-detection aggressive
```

Objetivo y contexto: Enumera plugins vulnerables (vp) y usuarios (u). La detección agresiva encuentra plugins que la pasiva no ve. Es el primer paso siempre en WordPress.

Resultado esperado: Version del core, plugins con su version y [!] de vulnerabilidad, y la lista de usuarios. Cada plugin vulnerable -> busca su exploit; cada usuario -> candidato a login.

Anatomía de los payloads

La primera prueba es `wpscan --url $URL --enumerate vp,u --plugins-detection aggressive`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Version del core, lista de plugins con su version, y usuarios enumerados via `/?author=` o REST API.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera core, plugins, temas y usuarios con wpscan (usa `--api-token` para ver CVEs).
- Cruza cada plugin/version con exploits conocidos; prioriza los pre-auth.
- Si no hay CVE, ataca el login (usuarios enumerados + passwords) con cuidado de bloqueos.
- Con acceso admin, edita un .php del tema para lograr ejecución y salta a shell (`1fi-a-rce` comparte la idea: escribir código que el servidor ejecuta) y luego `reverse-vs-bind`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- wpscan no detecta ningún plugin con CVE y el core está en la última versión.
- No hay usuarios enumerables ni login accesible para spraying.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El core suele estar parcheado; el agujero casi siempre está en un plugin o tema vulnerable, o en el panel de admin. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (wordpress)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Cross-Site Scripting (XSS)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un XSS ocurre cuando la app mete tu entrada dentro del HTML de una pagina sin neutralizar caracteres especiales (<, >, "), y el navegador de OTRA persona (o el tuyo) interpreta esa entrada como código HTML/JavaScript en vez de texto. Hay tres tipos: reflejado (el payload viaja en la URL/peticion y se refleja en la respuesta inmediata), almacenado (el payload se guarda en la BD y se sirve a cualquiera que vea esa pagina), y DOM-based (el JavaScript del CLIENTE inserta el payload sin pasar por el servidor).

El HTML no distingue 'texto que el usuario escribió' de 'código que el navegador debe ejecutar' salvo que la app escape los caracteres especiales antes de insertarlo. Es la misma causa raíz que `sqli` (mezclar dato y código), aplicada al navegador en vez de a la base de datos.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier campo cuyo valor termine mostrandose en HTML: comentarios, nombres de perfil, búsquedas, mensajes de error que reflejan tu input, parametros de URL que se pintan en la pagina.

Cómo identificarla

- Tu entrada aparece reflejada tal cual en el HTML de la respuesta (view-source lo confirma).
- Un campo de comentario/perfil que otros usuarios ven, sin indicacion de que se sanea.
- El payload de prueba dispara el `alert/console.log`.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Inyecta un payload minimo y visible, como `<script>alert(1)</script>` o `<img src=x onerror=alert(1)>`, en el campo sospechoso.

Evidencia esperada: Un dialogo de alerta ejecutandose en el navegador (o `console.log` visible en devtools) confirma que tu HTML/JS se interpreto, no se mostro como texto.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/search?q=<script>alert(1)</script>"
```

Objetivo y contexto: Prueba rápida por curl para ver si el payload se refleja SIN escapar en el HTML de respuesta, antes de abrir un navegador.

Resultado esperado: Busca `<script>alert(1)</script>` literal en el HTML devuelto (no `<script>`). Si aparece sin escapar, confirma en un navegador real para ver la ejecución.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/search?q=<script>alert(1)</script>"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un diálogo de alerta ejecutándose en el navegador (o `console.log` visible en devtools) confirma que tu HTML/JS se interpreta, no se muestra como texto.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica donde se refleja o almacena tu entrada viendo el HTML fuente (no solo lo que se ve renderizado).
- Prueba un payload mínimo de detección (`<script>alert(1)</script>`); si se filtra, prueba variantes (atributos de evento, distintas etiquetas).
- Distingue el tipo: si el payload va en la URL y se refleja al momento, es reflejado; si persiste tras recargar o lo ve otro usuario, es almacenado.
- En un CTF/lab, el impacto típico es robar la cookie de sesión de otro usuario o forzar una acción en su nombre.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app escapa los caracteres especiales (`<` se convierte en `<`) antes de insertar tu texto: aparece como texto literal, no se ejecuta.

- Un Content-Security-Policy estricto bloquea la ejecución de scripts inline, incluso si el HTML se inyecta.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada se inserta en el HTML/JS de la página sin escapar; un navegador que la visita ejecuta el JavaScript que tu pusiste. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(xss\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

XML External Entity (XXE)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

XML permite definir 'entidades' (variables de texto) en un DOCTYPE, incluidas entidades que cargan contenido EXTERNO (un fichero local o una URL). Si la app procesa XML que tu controlas y el parser no desactiva esas entidades externas, defines una que apunte a /etc/passwd y el parser te devuelve su contenido.

Muchos parsers XML traen la resolución de entidades externas activada POR DEFECTO por compatibilidad, y el programador no la desactiva explícitamente. El parser confía en que el XML que procesa es de confianza, pero si TU controlas el XML de entrada, controlas sus entidades.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Solo aplica cuando la app procesa XML que tu controlas: subida de ficheros XML/SVG/DOCX/XLSX, SOAP, SAML, o cualquier import que acepte XML. No aplica si la app solo usa JSON.

Cómo identificarla

- La app acepta subida o entrada de XML, SVG, SOAP, SAML o documentos Office (DOCX/XLSX usan XML internamente).
- El fichero XML que subes se procesa (parsea) en vez de guardarse tal cual.
- Errores de parser XML visibles en la respuesta ante un XML malformado.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Sube un XML con una entidad que apunte a un fichero que sepas que existe (file:///etc/passwd) y mira si el contenido vuelve reflejado.

Evidencia esperada: El contenido de /etc/passwd aparece en la respuesta (XXE 'visible'). Si no se refleja nada, intenta un XXE ciego (out-of-band) apuntando la entidad a tu propio listener HTTP.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
printf '<?xml version="1.0"?><!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>' > xxe.xml
```

Objetivo y contexto: Construye el XML mínimo: declara una entidad externa 'xxe' que apunta a un fichero local, y la referencia dentro del documento.

Resultado esperado: El fichero xxe.xml listo para enviar. El contenido de /etc/passwd debería sustituir a &xxe; si el parser resuelve la entidad.

Capa 2: prueba documentada

```
curl -sS -X POST $URL/upload --data-binary @xxe.xml -H 'Content-Type: application/xml'
```

Objetivo y contexto: Envía el XML malicioso al endpoint que lo procesa. El Content-Type application/xml asegura que el servidor lo trate como XML y no como texto plano.

Resultado esperado: Si el parser es vulnerable, la respuesta contiene las líneas de /etc/passwd (root:x:0:0...) en vez de procesar el documento normalmente.

Anatomía de los payloads

La primera prueba es `printf '<?xml version="1.0"?><!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>' > xxe.xml`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El contenido de /etc/passwd aparece en la respuesta (XXE 'visible'). Si no se refleja nada, intenta un XXE ciego (out-of-band) apuntando la entidad a tu propio listener HTTP.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma que el punto de entrada acepta y PROCESA XML (no solo lo almacena).
- Prueba una entidad local inocua (file:///etc/passwd en tu lab) para ver si se refleja el contenido.
- Si no hay reflexión, monta un XXE ciego: la entidad apunta a tu servidor HTTP/DNS y confirmas por el callback que llega.
- Con lectura de ficheros confirmada, busca configs con credenciales o claves SSH del servidor.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app solo acepta JSON o no procesa XML en ningún punto de entrada.
- El parser tiene DTD/entidades externas desactivadas (comportamiento por defecto en muchas librerías modernas parcheadas).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un parser XML mal configurado resuelve entidades externas que tú defines, dejándote leer ficheros del servidor o hacer peticiones salientes. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (xxe)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.